



TRABAJO FIN DE MÁSTER
Máster Inteligencia Artificial

Generación automática de procesos BPMN mediante inteligencia artificial y DSL

Estudiante: Ignacio González Peris
Dirección: Vicente Octavio Herrera García
Diego Canales Aguilera

Sevilla, Junio de 2026.

Resumen

El auge reciente de los Modelos de Lenguaje de Gran Escala (LLM), como ChatGPT, Claude o Gemini, ha impulsado la automatización del modelado de procesos de negocio a partir de lenguaje natural. Sin embargo, convertir directamente una descripción en una representación Business Process Model and Notation (BPMN) 2.0 presenta dificultades sintácticas, estructurales y semánticas, sobre todo cuando la salida requiere formatos extensos, con un elevado número de compuertas y nivel de anidamiento.

La hipótesis central de este trabajo sostiene que una representación intermedia basada en un lenguaje específico de dominio (*Domain-Specific Language*, DSL) proporciona un mejor rendimiento, en términos de robustez y calidad estructural, que la generación directa de BPMN-XML a partir de un *prompt*. Para contrastar esta hipótesis, se diseña e implementa un *pipeline* en el que el LLM genera un DSL textual simplificado. A continuación, un compilador determinista analiza la representación intermedia, valida sus restricciones y produce el BPMN 2.0 final. La evaluación principal examina la robustez, la calidad estructural, la cobertura de características semánticas y la estabilidad del enfoque DSL mediante quince procesos sintéticos y tres modelos, con una matriz experimental equilibrada de 45 ejecuciones por modelo.

Los resultados reflejan tasas elevadas de éxito semántico y de renderizado, además de diagramas generalmente sólidos en cuanto a conectividad y completitud estructural, lo que respalda la hipótesis de partida. A medida que aumenta la dificultad del proceso, la calidad disminuye, sin embargo, el sistema mantiene la capacidad de generar diagramas sintácticamente válidos. Aunque GPT-5.5 alcanza la media descriptiva más alta, las pruebas estadísticas no identifican diferencias significativas entre los tres modelos comparados.

Como ampliación experimental, el enfoque DSL se compara con la generación directa de BPMN-XML en condiciones *zero-shot*. La salida XML presenta una calidad elevada en los casos válidos, pero su calidad efectiva se reduce cuando se penalizan los fallos, especialmente en los procesos de estrés, lo que refuerza el valor de la representación intermedia frente a la generación directa.

Summary

Abstract

The recent rise of Large Language Models (LLMs), such as ChatGPT, Claude, and Gemini, has encouraged the automated generation of business-process models from natural-language descriptions. However, translating a narrative directly into Business Process Model and Notation (BPMN) 2.0 entails syntactic, structural, and semantic difficulties, especially when the required output is verbose and deeply nested.

The central hypothesis of this work is that using an intermediate representation through a Domain-Specific Language (DSL) yields better performance, in robustness and structural quality, than generating BPMN-XML directly from a prompt. To test it, this thesis designs and implements a pipeline in which the LLM generates a simplified textual DSL. A deterministic compiler parses this intermediate representation, validates its constraints, and produces the final BPMN 2.0 model. The main evaluation studies the robustness, structural quality, expected-feature coverage, and stability of the DSL approach over fifteen synthetic processes and three models, using a balanced experimental matrix of 45 runs per model.

The results show high semantic- and render-success rates and generally sound diagrams in terms of connectivity and structural completeness, supporting the initial hypothesis. Quality decreases as process difficulty increases, while the main omissions concern collaborations, pools, and message flows. Although GPT-5.5 achieves the highest descriptive mean, the statistical tests do not identify significant differences among the three compared models.

As an experimental extension, the thesis compares the DSL approach with direct BPMN-XML generation under zero-shot conditions. Direct XML achieves high quality among valid outputs, but its effective quality decreases when failed generations are penalized, especially for stress processes, which reinforces the value of the intermediate representation over direct generation.

Palabras clave:

- Business Process Model and Notation (BPMN)
- Modelo de lenguaje de gran escala (LLM)
- Lenguaje específico de dominio (DSL)
- Ingeniería de prompts
- Generación de procesos
- Evaluación automática
- Lenguaje natural
- Inteligencia Artificial
- BPMN Sketch Miner
- BEF4LLM

Keywords:

- Business Process Model and Notation (BPMN)
- Large Language Model (LLM)
- Domain-Specific Language (DSL)
- Prompt engineering
- Process generation
- Automated evaluation
- Natural language
- Artificial Intelligence
- BPMN Sketch Miner
- BEF4LLM

Índice general

1	Introducción	1
1.1	Contexto del problema	1
1.2	Motivación	2
1.3	Conceptos básicos de BPMN	2
1.4	Formulación del problema	4
1.5	Contribuciones	5
1.6	Estructura del documento	6
2	Contexto y estado del arte	8
2.1	Introducción al modelado de procesos conversacional	8
2.2	Evolución tecnológica: de enfoques tradicionales a LLM	9
2.3	El desafío de la generación directa y la verbosidad del XML	10
2.4	Representaciones intermedias y lenguajes de dominio específico (DSL)	10
2.5	Estrategias avanzadas de ingeniería de prompts	11
2.6	Orquestación multiagente (MAO) y frameworks MOSAIC	11
2.7	Evaluación de la calidad: el sistema 3C y verificación simbólica	12
2.8	Conclusiones y futuro del modelado con IA	13
3	Metodología	14
3.1	Selección de modelos	14
3.2	Modalidad de acceso: interfaz web frente a acceso programático	15
3.3	Compensación: réplicas múltiples	16
3.4	<i>System prompt</i> como variable controlada	16
3.5	Postprocesado mínimo	16
3.6	Ausencia de bucle cerrado de corrección	16
4	Desarrollo del sistema	18
4.1	Visión general: un monorepo con un motor compartido	18
4.2	El motor bpmn-core: de la descripción textual al diagrama	19
4.3	El DSL como representación intermedia legible	20
4.3.1	Análisis léxico y sintáctico	22
4.3.2	Análisis semántico	22
4.4	Emisión del XML BPMN	23

4.5	Subsistema de <i>layout</i> y renderizado	23
4.6	Validación y diagnósticos	24
4.7	Desarrollo por hitos y diagramas <i>canon</i>	25
4.8	Evolución del <i>system prompt</i> y mapeo <i>pool-lane</i>	25
4.9	Uso revisado de IA generativa y validación de extremo a extremo	26
4.10	Aplicaciones construidas sobre el motor	26
4.10.1	apps/tfm-lab: laboratorio de investigación	26
4.10.2	apps/company-web: aplicación probada en beta	26
4.11	Validación temprana con usuarios reales y colaboración con Metadev	27
4.12	Pipeline de evaluación y suite de pruebas	27
4.13	Síntesis	28
5	Evaluación	29
5.1	Objetivo de la evaluación	29
5.2	Diseño experimental	30
5.3	Métricas utilizadas	31
5.3.1	Relación con BEF4LLM	32
5.3.2	Robustez	34
5.3.3	Calidad estructural	34
5.3.4	Cobertura semántica	35
5.3.5	Consumo estimado de tokens	35
5.3.6	Diagnósticos del motor	36
5.4	Visión general del dataset	36
5.5	RQ1: Robustez del enfoque DSL	37
5.6	RQ2: Comparación de calidad entre modelos	38
5.7	RQ3: Efecto de la dificultad del proceso	40
5.8	RQ4: Adecuación semántica	41
5.8.1	Cobertura de características esperadas	41
5.8.2	Taxonomía de diagnósticos	42
5.9	RQ5: Sensibilidad al prompt de sistema	43
5.10	RQ6: Sensibilidad a la longitud de la descripción (planteamiento y trabajo futuro)	44
5.11	RQ7: Estabilidad entre ejecuciones	45
5.12	RQ8: Comparación <i>zero-shot</i> con generación BPMN-XML directa	46
5.13	Validez externa: procesos reales en español	49
5.14	Significancia estadística	50
5.15	Amenazas a la validez	51
5.15.1	Validez interna	51
5.15.2	Validez de constructo	51
5.15.3	Validez externa	51
5.15.4	Validez estadística	52
5.16	Reproducibilidad	52
5.17	Conclusiones de la evaluación	53

6 Conclusiones y Trabajo Futuro	55
6.1 Conclusiones	55
6.2 Trabajo futuro	57
6.2.1 Mejora de la colaboración entre participantes	57
6.2.2 Refinamiento del <i>system prompt</i>	57
6.2.3 Bucle cerrado de autorreparación	57
6.2.4 Ampliación de la comparación <i>zero-shot</i>	57
6.2.5 Evaluación semántica con referencia	58
6.2.6 Verificación simbólica y orquestación	58
6.2.7 Ampliación de la validez externa	58
6.2.8 Acceso mediante API e hiperparámetros controlados	58
6.2.9 Ampliación del conjunto de modelos	58
7 Organización del repositorio	59
7.1 Consolidación en un monorepo	59
7.2 Integración del paquete de evaluación	59
7.3 Documentación y automatización	60
Lista de acrónimos	61
Bibliografía	62
A Material adicional BPMN	67
A.1 Especificación del DSL y <i>system prompt</i>	67
A.2 Resultados completos y trazabilidad experimental	68
A.3 Ejemplo de extremo a extremo	68
A.4 Galería de diagramas <i>canon</i>	71
A.5 Catálogo de códigos de diagnóstico	71
A.6 Evolución del <i>system prompt</i>	73
A.7 Conjunto de procesos sintéticos	73

Índice de figuras

1.1	Leyenda de los elementos y símbolos básicos empleados en la notación estándar BPMN 2.0.	3
1.2	Ejemplo de diagrama BPMN 2.0 correspondiente al proceso de incorporación de un nuevo empleado (<i>onboarding</i>). Muestra la estructura secuencial, las pasarelas de decisión y la organización en carriles asignados a los departamentos. . .	4
1.3	Interfaz de usuario de <i>CompanyWeb</i> , aplicación web de demostración desarrollada para facilitar el modelado conversacional y la documentación de procesos con asistencia de Inteligencia Artificial (probada en <i>SGS Productivity</i>).	6
4.1	Arquitectura del sistema: del lenguaje natural a un modelo BPMN 2.0 renderizable. El LLM externo genera el DSL textual intermedio, que un compilador determinista transforma mediante las etapas <code>parseDsl</code> , <code>emitBpmnXml</code> , <code>renderSemanticXml</code> y <code>validateBpmnModel</code> . La generación es <i>single-shot</i> : el motor no reinyecta diagnósticos al modelo.	20
4.2	Diagrama BPMN 2.0 generado a partir del ejemplo <i>canon-1</i>	22
5.1	Robustez por modelo	37
5.2	Media de calidad estructural por modelo	38
5.3	Distribución del número de tareas y de compuertas por modelo. Los tres modelos presentan una verbosidad similar, con algún valor atípico en procesos complejos.	39
5.4	Relación entre el número total de nodos y la complejidad de flujo de control de los diagramas generados, por modelo y nivel de dificultad.	39
5.5	Número medio de tareas y de compuertas por modelo y nivel de dificultad. El tamaño del diagrama crece de forma marcada en el nivel de estrés.	40
5.6	Submodelado en procesos complejos	41
5.7	Ablación de cobertura semántica por versión del <i>system prompt</i>	44
5.8	Robustez, calidad condicional y calidad efectiva en la comparación global	46
5.9	Tasa de éxito semántico del DSL y del XML directo por dificultad	47
5.10	Calidad condicional y penalizada por dificultad	48
5.11	Asimetría entre tokens de entrada y salida según la representación	49
A.1	Modelo BPMN de la planta de tratamiento de residuos.	70

Índice de tablas

2.1	Comparación técnica entre Fine-tuning y Prompt Engineering	9
4.1	Construcciones principales del DSL intermedio.	21
5.1	Preguntas de investigación de la evaluación	30
5.2	Niveles de dificultad del corpus sintético	31
5.3	Subconjunto operativo de métricas inspirado en BEF4LLM	33
5.4	Métricas de robustez del pipeline	34
5.5	Distribución de estados de compilación del subconjunto DSL v3.1	36
5.6	Modelos con cobertura experimental completa	37
5.7	Tasa de éxito semántico y de renderizado (%) por modelo. Las tres configuraciones superan el 95 %, sin diferencias relevantes en robustez.	37
5.8	Calidad estructural media por modelo	38
5.9	Efecto de la dificultad del proceso	40
5.10	Cobertura media de características esperadas	41
5.11	Características esperadas omitidas con mayor frecuencia	42
5.12	Taxonomía de diagnósticos por modelo	43
5.13	Cobertura media de características por versión del <i>system prompt</i>	43
5.14	Estabilidad entre ejecuciones repetidas	45
5.15	Robustez, calidad y cobertura de GPT-5.5 por representación	46
5.16	Comparación DSL v3.1 frente a XML directo por dificultad	47
5.17	Estimación textual de tokens por generación con GPT-5.5	48
5.18	Prueba global de significancia estadística	50
5.19	Comparaciones par a par mediante Mann–Whitney U	50
5.20	Artefactos de reproducibilidad de la evaluación	52
A.1	Relación entre los conjuntos experimentales utilizados.	68
A.2	Diagramas canónicos utilizados como pruebas de regresión.	71
A.3	Diagnósticos del analizador léxico y sintáctico.	71
A.4	Diagnósticos del modelo y del subsistema de <i>layout</i>	72
A.5	Evolución funcional del <i>system prompt</i>	73
A.6	Organización del corpus sintético por dificultad.	74

Introducción

1.1 Contexto del problema: la evolución del modelado de procesos

El modelado de procesos de negocio constituye el núcleo de la agilidad organizacional, ya que permite traducir dinámicas operativas ambiguas o mal definidas en estructuras lógicas que favorecen la estandarización, optimización y la automatización. En la actualidad, la adopción de soluciones basadas en Inteligencia Artificial (IA) se ha convertido en una necesidad estratégica tanto para las empresas que pretenden reducir la fricción entre la concepción de un proceso y su ejecución, como para aquellas entidades que quieren estandarizar flujos de trabajo que ya se realizan en el seno de su organización. Tradicionalmente, el modelado con lenguajes de referencias como *Business Process Model and Notation* (BPMN) ha requerido el trabajo manual de analistas y un conocimiento técnico exhaustivo de las secuencias operativas del negocio, circunstancia que provoca numerosas reuniones en etapas iniciales del modelado. [1].

La aparición de los Modelos de Lenguaje de Gran Escala (*Large Language Models*, LLM), entre los que se encuentran familias como GPT, Claude y Gemini, ha revolucionado multitud de campos diferentes, sin embargo, en el ámbito del Business Process Management (BPM) la expectativa de convertir descripciones en lenguaje natural en modelos visuales y ejecutables no termina de alcanzar los niveles de analistas humanos [2]. La literatura científica advierte de desafíos críticos: los LLM encuentran dificultades significativas cuando intentan generar directamente archivos válidos para BPMN en lenguaje de marcado extensible (*eXtensible Markup Language*, XML), ya que incurren en errores estructurales que invalidan el modelo [1, 3]. Esta brecha técnica evidencia una paradoja: pese a que los procesos constituyen el activo más ubicuo de la organización, su formalización automatizada continúa siendo un reto técnico de primer orden debido a la verbosidad de XML.

1.2 Motivación: democratización y eficiencia en el diseño de procesos

Esta investigación se justifica por la exigencia de conocer detalladamente las cadenas de valor del negocio antes de proponer acciones de mejora y automatizaciones de extremo a extremo, así como la urgencia de democratizar el diseño de procesos y eliminar las barreras de entrada que encuentran los expertos de dominio sin formación técnica en notaciones formales. El desarrollo de herramientas de modelado conversacional, es un ámbito que en los últimos años se está empezando a abordar. Un caso concreto es BPMN-Chatbot++, que supone un avance extraordinario en la reducción de costes y tiempos de ciclo, al tiempo que mejora la accesibilidad mediante interfaces de lenguaje natural [4]. Estudios recientes comparan de forma directa el modelado asistido por *chatbots* generativos con el llevado a cabo por modeladores humanos, lo que permite delimitar las capacidades y los límites actuales de este enfoque [5]; en esta misma línea, se han propuesto marcos conversacionales que guían el modelado de procesos de manera incremental [6].

Desde el punto de vista de la arquitectura de sistemas, el uso del *in-context learning* a través de la ingeniería de *prompts* permite adaptar modelos generales sin necesidad de recurrir a un ajuste fino costoso [2] que suele provocar una reducción en la capacidad de generalizar del LLM en cuestión. En este trabajo, la capacidad generativa se integra con un componente simbólico determinista: el LLM genera una representación intermedia basada en un lenguaje específico de dominio (*Domain-Specific Language*, DSL), que el motor propio analiza, valida y transforma en BPMN 2.0. La implementación no abarca una arquitectura multiagente ni un verificador formal completo de *soundness* (solidez del comportamiento: ausencia de bloqueos y de partes muertas del flujo); su alcance se limita a comprobar en qué medida un DSL, un *system prompt* controlado y un compilador determinista permiten producir resultados reproducibles y evaluables de forma automática. Este primer paso, es fundamental para mejorar la calidad de sistemas más avanzados que puedan plantearse posteriormente.

1.3 Conceptos básicos de BPMN: notación y lectura de diagramas

Con el fin de facilitar la comprensión del trabajo a quienes no estén familiarizados con la notación de modelado de procesos, esta sección presenta de forma breve y resumida los símbolos clásicos de BPMN 2.0 y explica su interpretación. La notación estándar BPMN permite representar dinámicas operativas de manera gráfica y comprensible para todos los actores implicados mediante un conjunto de elementos visuales básicos.

Las cuatro categorías principales de elementos de flujo son las siguientes:

- **Eventos (Events):** Se representan mediante círculos y señalan un suceso que tiene lugar durante el desarrollo de un proceso. El evento de inicio presenta una línea fina; el evento intermedio, una línea doble; y el evento de fin, una línea gruesa. Existen numerosos eventos adicionales como de tiempo o timers.

- **Actividades (Activities/Tasks):** Se representan mediante rectángulos con bordes redondeados generalmente azules e indican un trabajo o una tarea específica que debe realizarse como parte del flujo del proceso. Pueden añadir símbolos para describir el tipo de tarea que se realiza, como por ejemplo manual versus automática.
- **Pasarelas (Gateways):** Se representan mediante rombos y funcionan como puntos de decisión que controlan la bifurcación, convergencia o división de los caminos del flujo de secuencia. Las más habituales son las pasarelas exclusivas (sin marcar o con una equis X), que abren caminos alternativos excluyentes, suelen tener una pregunta seguidas de Sí, No; y las pasarelas paralelas (marcadas con un signo más +), que dan inicio a actividades simultáneas o concurrentes.
- **Flujos de conexión (Connecting Objects):** Son flechas de línea continua que indican el orden secuencial de ejecución de las actividades dentro del proceso. En el caso de intercambio de mensajes, esta conexión se representa con líneas discontinuas.

La Figura 1.1 presenta una leyenda visual de los símbolos clásicos que sirven como referencia para interpretar la estructura de cualquier proceso representado con esta notación.



Figura 1.1: Leyenda de los elementos y símbolos básicos empleados en la notación estándar BPMN 2.0.

Además, los diagramas se organizan en contenedores denominados *Pools* (piscinas) y *Lanes* (carriles). Un *Pool* representa a un participante o a una entidad organizativa completa (por ejemplo, una empresa), mientras que los *Lanes* dividen el *Pool* horizontal o verticalmente para asignar responsabilidades a roles, departamentos o sistemas concretos de la organización. De manera que para una determinada acción del proceso no solo se puede apreciar su secuenciación y lógica, también se conoce quién la ejecuta.

Para ilustrar la interacción de estos componentes en un escenario real, la Figura 1.2 representa el modelado del proceso de incorporación de un nuevo empleado (*onboarding*). El

diagrama permite observar el inicio del proceso, la coordinación de las tareas distribuidas entre carriles asociados a distintos roles o departamentos (como Recursos Humanos y el supervisor directo), el uso de pasarelas para validar aprobaciones y la finalización exitosa del flujo.

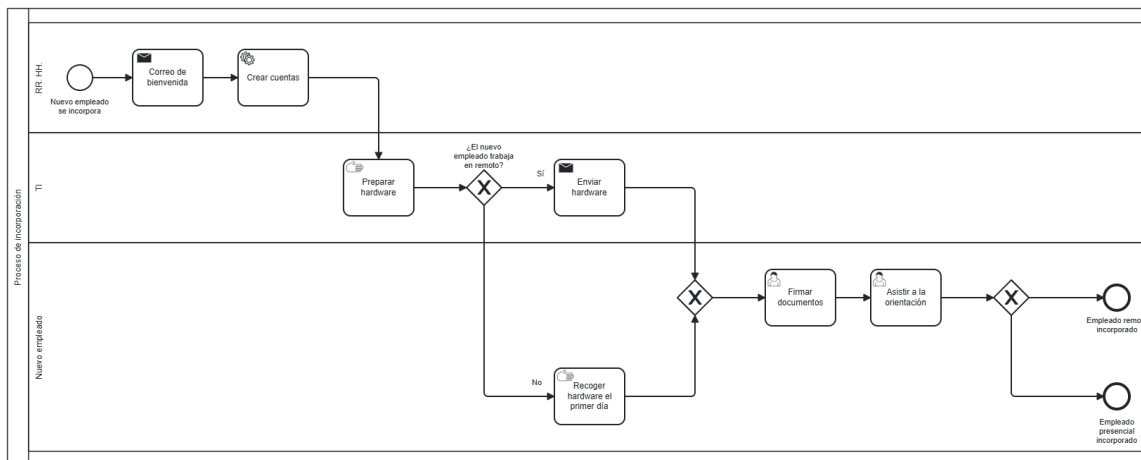


Figura 1.2: Ejemplo de diagrama BPMN 2.0 correspondiente al proceso de incorporación de un nuevo empleado (*onboarding*). Muestra la estructura secuencial, las pasarelas de decisión y la organización en carriles asignados a los departamentos.

1.4 Formulación del problema: el abismo entre el lenguaje natural y la sintaxis BPMN

El problema estudiado radica en la dificultad de los LLM para preservar al mismo tiempo la fidelidad semántica del proceso y las restricciones propias de una notación formal. La generación directa de XML puede dar lugar a errores sintácticos, referencias inconsistentes y estructuras de flujo incompletas [3]. Ante esta dificultad, la pregunta central de la investigación plantea si una representación intermedia, textual y restringida permite generar BPMN 2.0 con mayor robustez y calidad estructural que la generación directa de XML, así como la forma en que el resultado varía en función del modelo, la dificultad del proceso y el diseño del *prompt*.

Se adopta BEF4LLM [3] como marco de evaluación porque proporciona métricas objetivas y reproducibles sin requerir un diagrama de referencia. Otras propuestas recogidas en la literatura, Partially Ordered Workflow Language (POWL), el *framework* MOSAIC, el sistema 3C y los verificadores simbólicos, se analizan como antecedentes y alternativas [1, 7, 8], pero no se utilizan como base de la evaluación, ya que exigen comprobaciones más complejas o introducen un mayor grado de subjetividad en la valoración; ninguna de ellas corresponde a la arquitectura implementada. El sistema desarrollado utiliza una única generación del LLM, un DSL interno, un compilador determinista y métricas automáticas que no requieren un diagrama de referencia.

1.5 Contribuciones del trabajo

Este trabajo aporta las siguientes contribuciones originales al ámbito del modelado automatizado de procesos de negocio mediante Inteligencia Artificial:

1. **Evolución de un DSL basado en BPMN Sketch Miner hacia una representación propia.** La primera versión del sistema tomó como referencia inicial la sintaxis textual documentada por BPMN Sketch Miner [9]. Este enfoque se conservó, en sus aspectos esenciales, hasta el *system prompt* v3.1. Sobre esa base se ampliaron el DSL, el *parser* y el modelo semántico para atender necesidades que la notación original no contemplaba de manera directa. En v4, el *system prompt* incorporó conocimiento experto y criterios de diseño de procesos destinados a orientar la estructuración del BPMN, además de reglas de legibilidad y de tratamiento de eventos, de manera que entre otros aspectos, el LLM se asegurase de añadir información a los eventos de salida y entrada. Más adelante, v5 añadió la declaración explícita de *pools* y *lanes*, así como el soporte correspondiente en el analizador y el motor, lo que permite representar de forma controlada participantes independientes y sus interacciones, uno de los principales cuellos de botellas que presentaba el modelo en sus primeras versiones. La solución, por tanto, se inspira en BPMN Sketch Miner, pero no se limita a reproducirlo, pues incorpora capacidades propias en las instrucciones de modelado, la sintaxis, el modelo semántico y la generación final de BPMN.
2. **Diseño e implementación de un *pipeline* completo de generación y validación.** La cadena desarrollada consta de: (1) un LLM guiado por un *system prompt* estructurado, (2) un *parser* determinista que convierte el DSL en una representación interna, (3) las validaciones sintácticas y semánticas realizadas por el motor y (4) un emisor y renderizador de BPMN 2.0. Aunque estas validaciones permiten detectar defectos estructurales concretos y avisos por posibles malas prácticas, no equivalen a una prueba formal completa de *soundness*.
3. **Protocolo de evaluación reproducible y extensible.** La evaluación compara tres modelos, cada uno con 45 ejecuciones sobre quince procesos sintéticos y tres repeticiones. También incluye ablaciones de diferentes versiones del *system prompt* y una comparación *zero-shot* basada en la generación directa de XML.
4. **Despliegue en un entorno empresarial real.** Este trabajo no se limita a una evaluación mediante métricas académicas, también se valida el sistema en un contexto profesional real a través de su integración en CompanyWeb (Figura 1.3), una aplicación web de demostración probada por compañeros de SGS Productivity vinculados a la consultoría Lean y la mejora continua. Esta página web permite valorar no solo la precisión técnica del modelo generado, sino también la utilidad percibida por analistas y consultores que no siempre son expertos en notaciones formales. Aunque existen estudios de caso sobre el uso de LLM para modelar procesos en organizaciones [10], la evaluación con usuarios reales no expertos en entornos de producción sigue siendo escasa.

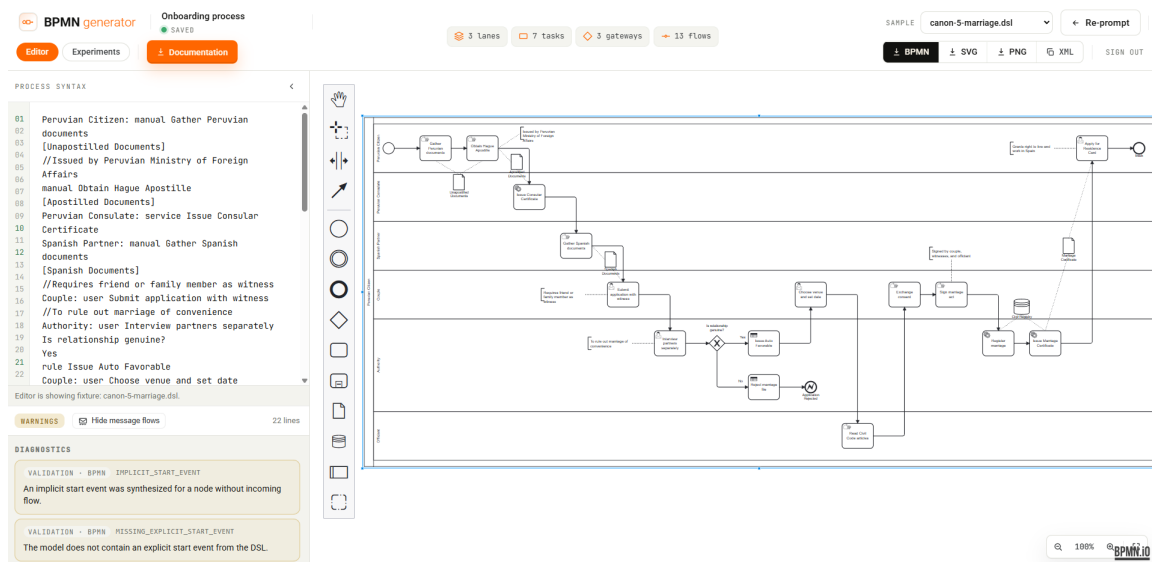


Figura 1.3: Interfaz de usuario de CompanyWeb, aplicación web de demostración desarrollada para facilitar el modelado conversacional y la documentación de procesos con asistencia de Inteligencia Artificial (probada en SGS Productivity).

La colaboración con la empresa tecnológica Metadev [11], además de motivar el correspondiente agradecimiento, representa una parte sustancial del valor metodológico de este trabajo. El soporte técnico y la mentoría proporcionados se han concretado en una supervisión y validación continuas por parte de expertos del sector durante todo el ciclo de elaboración de este Trabajo de Fin de Máster (TFM), muy especialmente en la especificación, el diseño y la validación de la gramática del DSL empleado como representación intermedia de los procesos. Por ello, esta validación experta refuerza la solidez de la contribución y no se reduce a un mero reconocimiento.

1.6 Estructura del documento: hoja de ruta de la investigación

Para facilitar el seguimiento de la progresión lógica de la tesis, el documento se estructura de la siguiente forma:

1. **Estado del Arte:** Se presenta un análisis crítico de los LLM en BPM y se evalúan técnicas de *prompt engineering* ante la escasez de datos existente en el sector.
2. **Metodología:** Se describe el *framework* propuesto y se justifica el uso de un DSL para garantizar la solidez estructural y la validez sintáctica.
3. **Desarrollo del sistema:** Se expone la arquitectura del monorepo, el motor `bpmn-core` para la transformación `DSL→BPMN` y los subsistemas de *layout*, validación y diagnóstico.
4. **Evaluación y Resultados:** Se realizan pruebas empíricas de robustez, calidad estructural, cobertura semántica y estabilidad, tomando como referencia el marco BEF4LLM [3].

5. **Conclusiones y Trabajo Futuro:** Se sintetizan los hallazgos y se plantean nuevas líneas para aplicar la IA generativa a los procesos de negocio.
6. **Organización del repositorio:** Se aborda la evolución del proyecto, su adopción en SGS Productivity y la integración del paquete de evaluación para completar el ciclo de reproducibilidad.

Esta estructura conduce a la revisión de la literatura técnica, necesaria para comprender la sólida base teórica y tecnológica que sustenta la propuesta.

Contexto y estado del arte

En este capítulo se revisa el estado del arte sobre la generación automatizada de modelos BPMN mediante modelos de lenguaje de gran escala (LLM). El análisis comprende desde el modelado conversacional hasta los marcos de evaluación de la calidad que permiten contextualizar la propuesta desarrollada en este trabajo.

2.1 Introducción al modelado de procesos conversacional

La disciplina del BPM atraviesa una etapa de democratización técnica sin precedentes. Tradicionalmente, la captura de flujos de trabajo requería la intervención de analistas especializados en notaciones complejas, circunstancia que favorecía la aparición de silos de conocimiento. La incorporación de los LLM ha permitido avanzar hacia el Modelado de Procesos Conversacional, un paradigma en el que la edición manual de diagramas se sustituye por una interacción dialógica en lenguaje natural. Diversos estudios empíricos recientes confirman que los LLM pueden abordar diversas tareas de BPM, entre ellas la generación de modelos y la minería de procesos consciente de la semántica, sin necesidad de entrenamiento específico [12, 13]. Esta capacidad ha impulsado la propuesta de *Large Process Models*, que combinan dichos modelos con conocimiento estructurado de procesos [14]. Otras líneas de investigación se orientan específicamente a democratizar el modelado mediante la generación de BPMN y la comprobación de su conformidad con guías de estilo [15].

Los estudios fundamentales sobre herramientas como ProMoAI [1] y BPMN-Chatbot++ [4] muestran que este enfoque permite tanto sistematizar procesos internos como ampliar su aplicación a colaboraciones inter-organizacionales [4]. De este modo, se facilita la gestión de diagramas de colaboración que incorporan múltiples pools y flujos de mensajes. Esta línea de asistentes conversacionales se prolonga en propuestas como BPMN Assistant, que utiliza un LLM para crear y editar modelos BPMN de manera interactiva [16], así como en enfoques dirigidos a la creación conversacional de modelos accionables [17]. Entre los beneficios señalados por la comunidad científica se encuentran los siguientes:

- **Accesibilidad para expertos de dominio:** Los dueños de los procesos pueden crear modelos sin contar con formación técnica previa sobre la sintaxis XML de BPMN.

- **Eficiencia operativa:** El tiempo necesario para pasar de una descripción narrativa a un modelo ejecutable se reduce de forma drástica.
- **Flexibilidad evolutiva:** Los modelos pueden refinarse de manera iterativa mediante una retroalimentación conversacional continua.

2.2 Evolución tecnológica: de enfoques tradicionales a LLM

Las técnicas de extracción de procesos han evolucionado desde métodos de procesamiento del lenguaje natural (*Natural Language Processing*, NLP), sustentados en reglas y en la minería de texto tradicional, hasta el uso de arquitecturas de *transformers*. Esta evolución ha permitido pasar de un tratamiento meramente sintáctico a la comprensión semántica del contexto de negocio. Los primeros sistemas destinados a generar modelos a partir de texto se basaban en reglas lingüísticas y análisis sintáctico, por lo que ofrecían una cobertura limitada y dependían en gran medida de la redacción utilizada [18]. Esta dificultad impulsó la creación de *benchmarks* específicos para extraer procesos a partir de texto [19]. La brecha se ha reducido progresivamente con métodos posteriores basados en NLP [20] y, más recientemente, con sistemas que automatizan la generación de extremo a extremo a partir de requisitos textuales [21].

De acuerdo con Busch et al. [2], el paradigma actual se articula en torno a dos alternativas: el ajuste fino y el aprendizaje en contexto. Dentro de la vertiente orientada a la especialización, algunos trabajos recientes ajustan LLM para el modelado de procesos mediante aprendizaje por refuerzo con recompensas verificables [22]. La Tabla 2.1 resume las principales diferencias técnicas entre ambas alternativas:

Característica	Fine-tuning (Ajuste Fino)	Prompt Engineering (In-context Learning)
Enfoque	Los parámetros se actualizan con datos etiquetados.	El modelo permanece «congelado» y se guía mediante instrucciones.
Coste computacional	Es elevado, pues requiere un entrenamiento intensivo.	Es bajo, al estar optimizado para una inferencia inmediata.
Huella de carbono	Resulta significativa debido a la frecuencia del reentrenamiento.	Es reducida y más sostenible.
Requerimientos de datos	Requiere grandes volúmenes de datos específicos.	Funciona eficazmente con pocos ejemplos (Zero/Few-shot).
Flexibilidad	Ofrece una alta especialización en una única tarea.	Aporta versatilidad para múltiples tareas de BPM.

Tabla 2.1: Comparación técnica entre Fine-tuning y Prompt Engineering

2.3 El desafío de la generación directa y la verbosidad del XML

Una de las barreras críticas para alcanzar una automatización completa es la incapacidad de los LLM para generar directamente código BPMN XML válido. Lauer et al. [3] documentan que los modelos incurren de forma recurrente en errores estructurales, incluso cuando reciben instrucciones explícitas. En la misma línea, Hörner et al. [23] sistematizan los desafíos asociados a la generación directa de modelos BPMN 2.0 a partir de descripciones en lenguaje natural y plantean un marco acompañado de una evaluación de calidad. Existen también propuestas que afrontan este problema mediante arquitecturas multietapa guiadas por LLM [24].

El problema central se encuentra en la verbosidad del XML. Este formato añade un volumen excesivo de ruido sintáctico y consume con rapidez la ventana de contexto del modelo. La sobrecarga resultante reduce la capacidad de razonamiento lógico del LLM y, además, induce errores graves, como etiquetas mal cerradas, dependencias circulares, *deadlocks* o partes inalcanzables del flujo.

El siguiente fragmento ilustra esta clase de fallos mediante un BPMN-XML defectuoso característico de la generación directa. En él aparecen una etiqueta sin cerrar y un flujo de secuencia que hace referencia a un nodo inexistente, de manera que el modelo resultante es inválido o queda inconexo.

```
<bpmn:task id="Task_1" name="Validar pedido">
<!-- etiqueta <task> sin cerrar -->
<bpmn:sequenceFlow id="Flow_1"
    sourceRef="Task_1" targetRef="Task_99"/>
<!-- Task_99 no existe: referencia colgante -->
```

La recurrencia de estos errores estructurales, documentada también de forma sistemática por Lauer et al. [3], es precisamente la que motiva el uso de una representación intermedia más restringida.

2.4 Representaciones intermedias y lenguajes de dominio específico (DSL)

Antes de presentar las propuestas concretas, conviene definir el concepto de lenguaje específico de dominio (*Domain-Specific Language*, DSL). Frente a un lenguaje de propósito general, un DSL se diseña para un dominio delimitado y emplea una sintaxis y un vocabulario reducidos, adaptados a los conceptos propios de dicho ámbito. Esta especialización ofrece una representación más concisa y legible que la de un formato genérico como el XML. También permite limitar de antemano las construcciones admitidas, lo que simplifica tanto la generación por parte de un LLM como la validación automática.

Para reducir los problemas derivados de la verbosidad del XML, la investigación de vanguardia propone utilizar DSL y formatos estructurados intermedios:

- **JSON Estructurado:** BPMN-Chatbot++ lo utiliza para gestionar la complejidad asociada a pools, lanes y flujos de mensajes en escenarios inter-organizacionales [4].
- **POWL:** Este lenguaje, empleado por el *framework* ProMoAI, garantiza de manera intrínseca el *soundness* estructural. Una de sus características técnicas distintivas es la lógica de "paralelismo por defecto": las tareas se consideran concurrentes salvo que se establezcan dependencias explícitas [25], lo que optimiza la eficiencia de generación del LLM.

Ventajas de los DSL sobre XML: La abstracción proporcionada por un DSL reduce de forma drástica el consumo de tokens y permite validar las reglas de negocio de manera determinista. Con ello se garantiza la corrección estructural (*soundness*) y se simplifica la lógica que necesita el LLM para converger en un modelo válido. Esta idea dispone de respaldo empírico: estudios comparativos recientes indican que las representaciones textuales y los DSL son competitivos frente al XML como formato objetivo para la generación mediante LLM y que, en varias métricas, llegan a obtener resultados superiores [26, 27].

2.5 Estrategias avanzadas de ingeniería de prompts

La calidad del modelado automatizado depende de la sofisticación de las técnicas de *prompting* utilizadas para orientar al modelo [1, 28]:

1. **Role Prompting:** Consiste en asignar al LLM el rol de experto senior en BPM y dueño de proceso, con el fin de establecer el contexto semántico adecuado.
2. **Knowledge Injection:** La gramática técnica del DSL (como POWL o esquemas JSON) se incorpora directamente al prompt para limitar la salida al formato requerido.
3. **Few-shot Learning:** Se proporcionan ejemplos robustos, como el proceso de fabricación de bicicletas [1], para que el modelo aprenda la relación entre la descripción textual y la estructura lógica.
4. **Negative Prompting:** Se emplean instrucciones preventivas destinadas a evitar errores frecuentes, como las violaciones de irreflexividad en las dependencias o la generación de *local choices* entre actividades, en vez de elecciones entre rutas completas del proceso.

Además de estas técnicas específicas, se han planteado estrategias de *prompting* de propósito general para extraer, mediante distintos LLM, información sobre modelos de proceso a partir de texto [29].

2.6 Orquestación multiagente (MAO) y frameworks MOSAIC

En el ámbito del modelado de procesos han aparecido propuestas de orquestación multi-agente, entre ellas el *framework* MAO (*Multi-Agent Orchestration*, orquestación multiagente) [7]. En estos enfoques, los objetivos globales se dividen en sub-tareas que ejecutan agentes autónomos especializados. El paradigma MOSAIC [30] constituye un ejemplo de esta línea de trabajo.

La arquitectura se estructura de acuerdo con una jerarquía de roles formada por el Controlador, el Planificador, el Ejecutor (Worker) y el Evaluador. El flujo se gestiona mediante máquinas de estados finitos (FSM) y grafos acíclicos dirigidos (DAG), cuyo uso permite evitar el *unobservable state drift* (deriva de estado no observable).

Los *Quality Gates* (puertas de calidad) constituyen un componente esencial de esta arquitectura. En lugar de aplicar una decisión binaria, estos mecanismos clasifican los resultados en varias categorías operativas: *gate_done*, *gate_fail*, *gate_continue* o *gate_supplement*. Cuando el evaluador identifica una anomalía, el sistema activa funciones proactivas de corrección (*auto-fix*) para solucionar el error antes de producir la entrega final.

Estas arquitecturas se incluyen como parte del contexto del estado del arte y como posibles vías de evolución. El prototipo desarrollado en este TFM no implementa MOSAIC, agentes especializados, puertas de calidad iterativas ni un bucle de autocorrección. Se adopta deliberadamente una arquitectura más acotada, basada en una generación *single-shot* del DSL a la que siguen una compilación y una validación deterministas. Esta delimitación facilita la atribución de los errores al LLM o al motor.

2.7 Evaluación de la calidad: el sistema 3C y verificación simbólica

La validación de modelos generados mediante IA requiere marcos de evaluación capaces de integrar la perspectiva de negocio y el rigor técnico:

- **BEF4LLM:** plantea una evaluación multidimensional de los modelos BPMN generados por LLM a través de métricas de validez del fichero, calidad sintáctica, calidad pragmática y calidad semántica [3]. Varias de estas medidas son cercanas a las aplicadas en este trabajo, sobre todo las referidas a la validez BPMN-XML, la conectividad, el etiquetado, la complejidad y la estructura de los elementos. En el capítulo de evaluación se describen la adaptación específica del marco y las diferencias que se derivan de no contar con un diagrama de referencia.
- **Dimensiones 3C:** Drakopoulos et al. [8] estructuran la evaluación en tres dimensiones: Claridad (comprensión visual), Corrección (reglas sintácticas) y Completitud (cobertura del texto original).
- **IA Simbólica y Model Checking:** Con el objetivo de garantizar la integridad, los modelos se someten a verificadores simbólicos que se integran como servicios independientes

de Spring Boot dentro de un Service Registry [4]. Entre las principales herramientas se encuentran:

- **S³ (Soundness Checker)**: Se orienta específicamente a la detección de errores estructurales y *dead tokens*.
- **vPAV (viadee Process Application Validator)**: Se utiliza para analizar flujos de datos e identificar anomalías, como variables que se escriben pero nunca se leen.

Los resultados de la arquitectura híbrida de MAO muestran una efectividad notable. Según Lin et al. [7], este *framework* genera modelos cuya calidad técnica supera la de los diagramas elaborados por humanos en un rango del 52 % al 89 %.

Los verificadores S³ y vPAV, así como POWL y el sistema 3C, no se incluyen en la implementación evaluada. Su presentación permite situar este trabajo frente a soluciones que incorporan comprobaciones de mayor alcance. Estos enfoques se descartan como marco de evaluación en favor de BEF4LLM, ya que requieren una infraestructura de verificación más compleja (S³, vPAV) o incorporan un mayor grado de subjetividad en la valoración (3C). En cambio, BEF4LLM proporciona métricas objetivas, reproducibles y aplicables sin necesidad de disponer de un diagrama de referencia. Las validaciones desarrolladas en este trabajo comprueban la conectividad, los grados, la completitud de las decisiones, el etiquetado, el emparejamiento de compuertas, la cobertura de características y los diagnósticos del motor. Sin embargo, no permiten afirmar formalmente que no existan bloqueos en todas las trazas posibles.

2.8 Conclusiones y futuro del modelado con IA

La literatura refleja una evolución hacia sistemas híbridos que integran generación, representaciones intermedias, validación y, en determinados casos, reparación iterativa. En este contexto, el TFM analiza un componente específico: la viabilidad de un DSL textual generado en una única interacción y posteriormente transformado mediante un compilador determinista. MOSAIC, POWL, 3C y los verificadores formales representan alternativas más ambiciosas, mientras que el trabajo experimental se centra en la robustez y la calidad del enfoque basado en DSL que se ha implementado.

Metodología

Este capítulo define un *pipeline* que parte de un *prompt* en lenguaje natural y, mediante varias etapas en las que interviene un LLM, genera un diagrama BPMN 2.0 válido. La idea de fondo es sencilla. En lugar de generar directamente el BPMN, el modelo produce una representación intermedia en un DSL textual; un compilador determinista la analiza y la valida antes de transformarla en el diagrama final. La Figura 4.1 (capítulo 4) presenta una visión global del flujo. En las secciones siguientes se describen sus distintas etapas y las restricciones metodológicas que las condicionan.

En la primera etapa del *pipeline*, una especificación informal del proceso se convierte en un documento conforme a un DSL interno, inspirado en la sintaxis textual documentada por BPMN Sketch Miner y compatible con ella [9]. De esa conversión se encarga un LLM condicionado mediante un *system prompt* fijo. En su versión v3.1, este recoge las reglas y limitaciones del DSL, los antipatrones, varios ejemplos completos y una lista final de comprobación; el Apéndice A reproduce su contenido íntegro. Este artefacto se conserva versionado en el repositorio del proyecto, en [apps/tfm-lab/prompts/system/bpmn_sketch_miner_system_prompt_v3_1.md](https://github.com/tfm-lab/prompts/system/bpmn_sketch_miner_system_prompt_v3_1.md) [31]. Su contenido parte de la sintaxis textual de BPMN Sketch Miner [9], que se amplió con conocimiento experto durante el desarrollo del trabajo. Este tipo de condicionamiento sin reentrenamiento se encuadra en el paradigma de *in-context learning*, introducido a gran escala por Brown et al. [32] y sistematizado posteriormente por Liu et al. [33].

3.1 Selección de modelos

La comparación principal incluye tres LLM disponibles para el usuario final: GPT-5.5 Thinking, Gemini 3.5 Flash Thinking y Claude Opus 4.7. Los tres eran representativos del estado del arte cuando se realizó este trabajo, en 2026. Dado el ritmo al que aparecen modelos nuevos, conviene leer esta selección dentro de ese contexto temporal. Para cada modelo se dispone de 45 ejecuciones realizadas en las mismas condiciones. La elección de los tres obedece a un propósito comparativo: caracterizar la variabilidad de los resultados según el proveedor, manteniendo constantes las demás variables. Opus 4.8 Medium solo dispone de una matriz parcial de ejecuciones v3.1; por ello, sus resultados se consideran exploratorios y no se integran en el

ranking principal.

3.2 Modalidad de acceso: interfaz web frente a acceso programático

Todos los modelos se invocaron mediante sus respectivas interfaces web públicas, en lugar de utilizar una interfaz de programación de aplicaciones (*Application Programming Interface*, API). Cabían dos formas de llevar a cabo el análisis, la interfaz web o la API, y se optó por la primera. Se trata de una decisión deliberada, fundamentada en un criterio de validez ecológica en el sentido clásico de Shadish, Cook y Campbell [34]. La validez ecológica expresa hasta qué punto las condiciones de un estudio se corresponden con los contextos reales de uso a los que se desean generalizar sus resultados. Cuanto mayor sea la semejanza entre las condiciones experimentales y el escenario real, mayor será dicha validez. En numerosos escenarios de consultoría de procesos, un perfil habitual es el del analista funcional que interactúa con el LLM mediante su *chat*, sin integrarlo a través de una API, aunque en otros contextos, como la integración de Metadev en su propia herramienta, el acceso sí se realiza por API. El estudio de la interfaz web permite así analizar uno de los flujos de trabajo más representativos. La comparación sistemática con el acceso mediante API queda reservada como trabajo futuro.

La adopción de esta modalidad supone renunciar expresamente a varios mecanismos de control experimental disponibles mediante API. En concreto:

1. **No hay control sobre los hiperparámetros de muestreo.** El proveedor establece internamente los valores efectivos de *temperature*, *top-p* y *random seed*, que el usuario no puede consultar.
2. **El identificador del modelo es un nombre comercial, no un *checkpoint* de pesos.** Las etiquetas utilizadas en el estudio, como «GPT-5.5 Thinking» o «Gemini 3.5 Flash Thinking», pueden corresponder a distintas versiones de los pesos en diferentes momentos, sin que el usuario reciba una notificación. Para mitigar este efecto, cada experimento registra la fecha de ejecución y la etiqueta exacta que muestra la interfaz. Ambos datos forman parte del identificador único del experimento (EXP-<proceso>-<sysprompt>-<modelo>-<versión>-R<n>).
3. **No existe API de *batching*.** Cada generación requiere una sesión manual. Esto limita el tamaño total del corpus, pero también garantiza que ninguna ejecución contamine las posteriores.
4. **Riesgo de filtración de contexto entre sesiones.** Para evitar este riesgo, cada par (proceso, modelo) se ejecuta en una conversación nueva y sin reutilizar el historial.

3.3 Compensación: réplicas múltiples

Sin control sobre la semilla aleatoria, la compensación llega por otra vía: réplicas independientes para cada celda (modelo $\times$ proceso), tres por celda en este trabajo (R01, R02, R03). Así, la varianza que introduce el muestreo estocástico interno del modelo se observa directamente, en lugar de eliminarla de forma artificial con una semilla fija. Dentro de las limitaciones del entorno, esta estrategia reproduce la práctica recomendada para evaluar la generación con LLM y se ajusta a la discusión de Sclar et al. [35] sobre la estabilidad de los *prompts*.

3.4 *System prompt* como variable controlada

En un experimento controlado conviene mantener constantes todas las variables, excepto aquella que se pretende estudiar. Como el modelo es la variable de interés de la matriz principal, el *system prompt* se considera una variable controlada y se fija en una única versión. Así, las diferencias observadas no pueden atribuirse a cambios en las instrucciones proporcionadas al LLM, sino al propio modelo, al proceso o a la variación entre réplicas. El *system prompt* empleado en la matriz principal es `bpmn_sketch_miner_system_prompt_v3_1.md`; para elaborar el *ranking* de modelos se filtran 135 ejecuciones equilibradas con la etiqueta SYSV31. La variable experimental `system_prompt_version` indica la versión del *system prompt* utilizada en cada ejecución y admite los valores v3.1, v4 y v5. Las versiones v4 y v5 se analizan de forma independiente en la ablación de RQ5, mediante matrices completas de 45 ejecuciones para GPT-5.5 y Gemini 3.5. Por tanto, `system_prompt_version` constituye una dimensión experimental explícita que no se combina con la comparación principal.

3.5 Postprocesado mínimo

La salida del LLM se proporciona al *lexer* prácticamente *verbatim*. La única transformación consiste en eliminar los delimitadores Markdown de tipo *fenced code block* (`````) cuando el modelo encierra el DSL entre ellos. Este postprocesado mínimo es deliberado, ya que cualquier transformación adicional difuminaría la separación entre el «comportamiento del LLM» y el «comportamiento del *pipeline*», lo que dificultaría la atribución de los errores durante la evaluación.

3.6 Ausencia de bucle cerrado de corrección

El LLM no recibe *feedback* del *parser* ni del renderizador. La ausencia de realimentación responde a una decisión metodológica deliberada. La matriz principal pretende medir la capacidad del modelo para producir un DSL correcto en una sola generación (*single-shot*), de manera que el resultado represente el comportamiento del LLM y no el de un mecanismo adicional de

reparación. Si los errores del motor se reinyectaran en el modelo, no sería posible determinar qué parte del éxito procede del propio modelo y cuál se debe a la corrección iterativa, lo que dificultaría la atribución de los fallos. Como contrapartida, los errores sintácticos no se corrigen automáticamente, sino que se contabilizan como fallos durante la evaluación. Esto proporciona una cota inferior conservadora del rendimiento que podría alcanzarse mediante realimentación. Cuando el DSL contiene errores sintácticos o incumple la regla de frontera de fragmentos descrita en la sección 12.1 del *system prompt*, antipatrones AP-6 y AP-7, el error se registra, pero no se reinyecta en el modelo. Ambos antipatrones se relacionan con una ubicación incorrecta de los marcadores de continuación de traza (...). El AP-6 ocurre cuando una traza regular finaliza en una tarea que también actúa como ancla de un fragmento, pero no incorpora un ... final; como resultado, se genera una compuerta espuria con un evento de enlace colgante. El AP-7 se da cuando varios fragmentos se anclan a una tarea ubicada dentro de otro fragmento, en lugar de hacerlo en una frontera ..., lo que produce eventos de enlace sueltos. La incorporación de un bucle de *self-repair*, en línea con Self-Refine [36] o PICARD [37], se reserva como trabajo futuro.

Desarrollo del sistema

Este capítulo recoge el sistema software construido durante el trabajo. El punto de partida es una descripción del proceso en lenguaje natural; a partir de ella, el LLM produce una representación intermedia en un DSL textual. Esa salida no se transforma directamente en un diagrama, sino que pasa por un compilador determinista que la analiza, la valida, genera el BPMN 2.0 correspondiente, aplica la maquetación automática y emite diagnósticos de calidad. El mismo conjunto de funciones, análisis del DSL, emisión de BPMN, maquetación y validación, se utiliza en tres contextos distintos: una aplicación de demostración para usuarios reales, el laboratorio experimental y el *pipeline* de evaluación. Para evitar divergencias entre estas superficies, el proyecto se organizó como un *monorepo*. Las secciones siguientes explican esta decisión y describen los componentes principales del sistema. La Figura 4.1 resume visualmente la relación entre ellos.

4.1 Visión general: un monorepo con un motor compartido

El resultado del desarrollo no es un *script* puntual, sino un pequeño ecosistema de herramientas que dependen de un mismo núcleo. Por ese motivo, la estructura del repositorio se planteó desde el principio como un *monorepo* gestionado mediante *pnpm workspaces*. En él se agrupan el paquete motor, las aplicaciones que lo consumen, el sistema de evaluación y los recursos auxiliares de documentación y orquestación:

```
packages/
  bpmn-core/      Motor compartido: DSL -> BPMN
apps/
  tfm-lab/        Laboratorio, experimentos y modo headless
  company-web/    Aplicacion web de demostracion
TFM-eval/         Pipeline de evaluacion en Python
docs/             Documentacion y decisiones de diseno
scripts/          Orquestacion de renderizado y evaluacion
```

El repositorio completo está disponible públicamente en <https://github.com/ignagp34/BPMN-DSL-Monorepo> [31] y funciona como fuente de verdad del proyecto. Las rutas citadas en este capítulo deben entenderse siempre con respecto a dicho repositorio.

La ventaja práctica de esta organización es que las mejoras del motor llegan a todas las superficies sin copiar lógica ni mantener variantes paralelas. Las aplicaciones solo consumen la API pública expuesta por `packages/bpmn-core/src/index.ts`. El resto de módulos, `dsl/`, `bpmn/`, `render/` y `validation/`, queda encapsulado como parte interna de la implementación.

Esta decisión también tiene una consecuencia metodológica importante. El diagrama que ve un usuario en la aplicación y el diagrama que se evalúa en los experimentos proceden del mismo código. Así se reduce el riesgo de validar una versión distinta de la usada en la demostración y se conserva una trazabilidad directa entre desarrollo, validación y uso real.

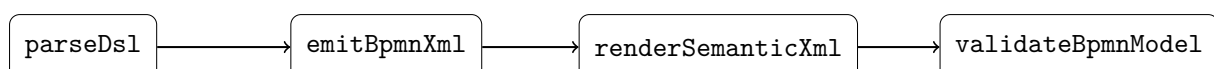
4.2 El motor bpmn-core: de la descripción textual al diagrama

El componente central del sistema es el motor `@text-to-bpmn/core`. Su tarea consiste en tomar una descripción textual escrita en un DSL propio, diseñado para ser más fácil de generar por un modelo de lenguaje que el XML BPMN completo, y convertirla en un diagrama BPMN 2.0 válido y maquetado. Para mantener controlado el proceso, la transformación se divide en cuatro etapas:

1. `dsl/`: análisis léxico y sintáctico, construcción del árbol de sintaxis abstracta (AST) y resolución semántica de roles, eventos implícitos y participantes.
2. `bpmn/`: emisión del XML BPMN a partir del modelo semántico resuelto.
3. `render/`: colocación de nodos, enrutado ortogonal, distribución de aristas, posicionamiento de etiquetas y organización de *pools* y *lanes*.
4. `validation/`: comprobaciones estructurales y generación de diagnósticos sobre el modelo y su *layout*.

Dividir el motor de esta forma facilita interpretar los fallos. Si el problema aparece en `dsl/`, la salida del modelo no ha podido interpretarse correctamente; si aparece en `render/`, el conflicto pertenece a la maquetación y no debería atribuirse a la capacidad del LLM para describir el proceso. Esta diferencia es la base de la separación entre `semantic_success` y `render_success` empleada en el Capítulo 5.

La API del motor refleja esa división. Cada fase puede invocarse de forma independiente y, además, existe el método de extremo a extremo `generateDiagram`, que encadena conceptualmente todo el flujo:



La Figura 4.1 muestra esta arquitectura de extremo a extremo. El recorrido empieza con la descripción del proceso en lenguaje natural y con el traspaso manual de la salida del LLM. No hay acceso por API ni bucle de corrección automática. A partir de esa salida, el compilador determinista `@text-to-bpmn/core` genera un modelo BPMN 2.0 renderizable.

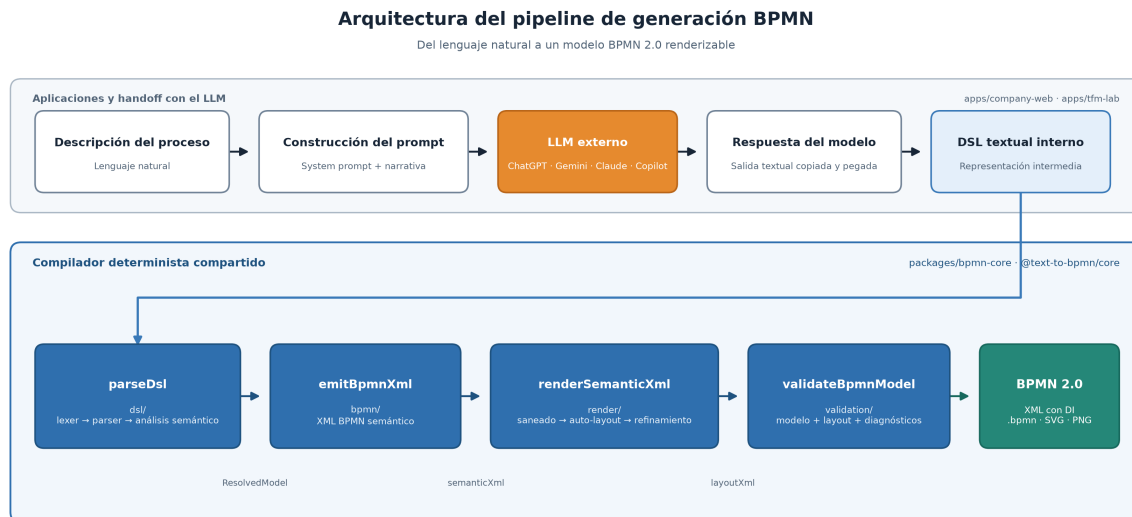


Figura 4.1: Arquitectura del sistema: del lenguaje natural a un modelo BPMN 2.0 renderizable. El LLM externo genera el DSL textual intermedio, que un compilador determinista transforma mediante las etapas `parseDsl`, `emitBpmnXml`, `renderSemanticXml` y `validateBpmnModel`. La generación es *single-shot*: el motor no reinyecta diagnósticos al modelo.

4.3 El DSL como representación intermedia legible

La elección de un DSL intermedio es una de las decisiones que más condiciona el enfoque del trabajo. No se trata solo de una capa técnica entre el LLM y el BPMN final: al ser textual, legible y verificable de forma determinista, permite inspeccionar la salida del modelo antes de llegar al diagrama. Gracias a ello puede distinguirse entre dos aspectos que, si se generara directamente XML BPMN, quedarían mezclados: lo que el LLM ha descrito correctamente y lo que el motor consigue maquetar. Esta capacidad de comprobación objetiva, ya anticipada en la introducción y en el estado del arte, sostiene el protocolo de medición automática presentado en el Capítulo 5.

En vez de pedir al modelo que escriba XML BPMN, se le solicita una representación textual y jerárquica. Cada línea describe un paso, y la posición de esa línea dentro de una traza marca el flujo de secuencia. La gramática aparece en el *system prompt* y se implementa en el analizador léxico. La Tabla 4.1 recoge las principales construcciones del DSL intermedio.

El ejemplo *canon-1*, almacenado en `packages/bpmn-core/tests/fixtures/canon-1-c-heeseburger.dsl`, ilustra el tipo de entrada que se busca: pocas líneas, legibles, pero con información suficiente para expresar roles, paralelismo entre estaciones de trabajo, un evento

Construcción	Sintaxis de ejemplo	Significado BPMN
Tarea	Hacer algo	Actividad.
Tarea con rol	Cajero: Tomar pedido	Actividad asignada a un <i>pool</i> o <i>lane</i> .
Tipo de tarea	service Validar pago	Tarea de usuario, servicio, regla, manual, recepción, envío o <i>script</i> .
Evento	(timer suena)	Evento intermedio de tiempo, mensaje, señal o error.
Inicio y fin	(start Inicio), (finish)	Eventos de inicio y fin.
Decisión	Pregunta? y sus ramas	<i>Gateway</i> exclusivo con ramas etiquetadas.
Paralelismo	A B C	Ramas concurrentes mediante <i>gateway</i> paralelo.
Datos	[Factura], [db Pedidos]	Objeto o almacén de datos.
Fragmento	...	Continuidad de una traza mediante anclas.
Mapa de <i>pools</i>	== pools ==	Declaración explícita de <i>pools</i> y <i>lanes</i> en V5.

Tabla 4.1: Construcciones principales del DSL intermedio.

de temporizador y eventos explícitos de inicio y fin. La Figura 4.2 muestra el diagrama BPMN 2.0 que el motor genera a partir de esa entrada DSL.

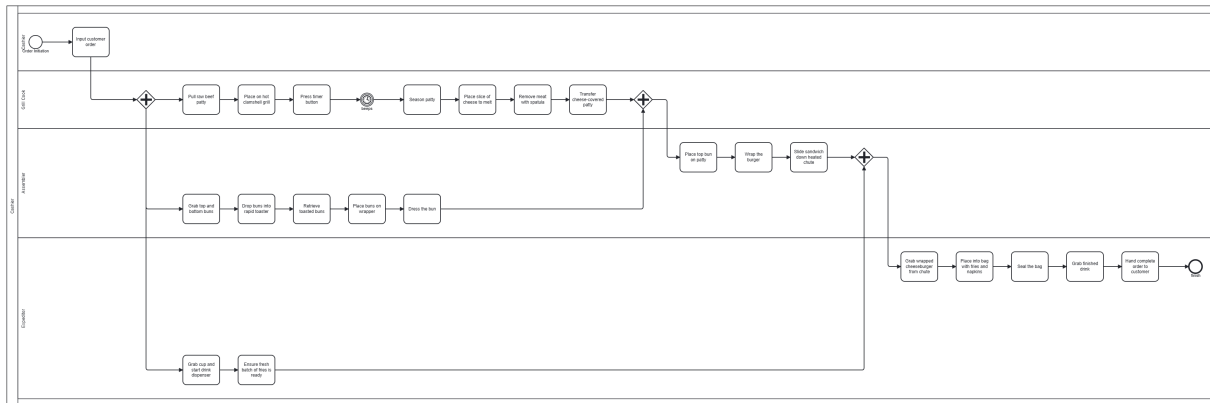


Figura 4.2: Diagrama BPMN 2.0 renderizado por el motor a partir del ejemplo *canon-1* (*canon-1-cheeseburger.dsl*). Ilustra roles, estaciones de trabajo en paralelo, un evento de temporizador y eventos explícitos de inicio y fin.

La representación interna se apoya en el AST y en el modelo resuelto definidos en `packages/bpmn-core/src/dsl/ast.ts`. Allí se distinguen pasos como Task, Event, Data, Question, Parallel, PoolScope, Annotation y FragmentMarker. Más adelante, estos elementos se convierten en nodos BPMN, flujos de secuencia, flujos de mensaje y participantes resueltos.

4.3.1 Análisis léxico y sintáctico

El análisis léxico y sintáctico se implementa con Chevrotain, principalmente en `lexer.ts` y `parser.ts`. La primera parte del proceso trabaja línea a línea: identifica si cada entrada corresponde a una tarea, un evento, un dato, una pregunta, una construcción paralela, una anotación, un fragmento o una cabecera del mapa de *pools*. A continuación, el *parser* agrupa esos pasos en trazas y construye el AST.

Las desviaciones se notifican mediante códigos estables, agrupados según su origen:

- análisis básico: LEX-1, PARSE-1, AP-6 y AP-7;
- anclaje de fragmentos: ANCHOR-MISSING, FRAGMENT-OPEN y FRAGMENT-DUP; y
- formato y estructura: INLINE-COMMENT y DANGLING-QUESTION.

Al usar códigos estables, los errores no quedan reducidos a mensajes textuales difíciles de comparar. La misma taxonomía sirve para localizar el problema durante el desarrollo y para reutilizar los diagnósticos en la evaluación.

4.3.2 Análisis semántico

Una vez construido el AST, la fase semántica lo convierte en un `ResolvedModel`. Esta lógica, implementada en `semantic.ts`, asigna identificadores únicos, resuelve los roles en *pool/s*

y *lanes*, conecta los flujos según el orden de las trazas, empareja las anclas y sintetiza los elementos implícitos que sean necesarios.

Un caso habitual es la ausencia de eventos explícitos de inicio o fin. En esas situaciones, el motor puede generarlos, pero registra la normalización mediante los diagnósticos `IMPLICIT_START_EVENT` e `IMPLICIT_END_EVENT`. La corrección, por tanto, no se oculta: queda trazada para su análisis posterior.

4.4 Emisión del XML BPMN

La generación del XML BPMN se concentra en el módulo `bpmn/`, formado sobre todo por `emit.ts` y `xml.ts`. Esta etapa produce un XML BPMN 2.0 todavía semántico, donde ya aparecen procesos, participantes, nodos de flujo, flujos de secuencia y de mensaje, objetos y almacenes de datos y eventos de frontera.

La distinción entre semántica y representación gráfica resulta especialmente útil en este punto. Primero se decide qué contiene el proceso; después, en otra fase, se decide dónde se dibuja cada elemento. Gracias a esta separación, el *layout* puede tratarse como una etapa independiente y sustituible, y el modelo puede validarse antes de asociarle una geometría concreta.

4.5 Subsistema de *layout* y renderizado

El *layout* concentró una parte importante del esfuerzo de ingeniería. Aunque el XML BPMN sea correcto, el resultado pierde valor si el diagrama no se puede leer con claridad. Por ello, el renderizado se organizó como un *pipeline* de pasos pequeños, cada uno con una responsabilidad concreta:

1. saneado del XML semántico para `bpmn-auto-layout`;
2. aplicación de un *auto-layout* base;
3. maquetación de procesos que todavía no disponen de información gráfica;
4. colocación de *pools* y *lanes* y preparación de los flujos de mensaje;
5. ortogonalización del flujo de control;
6. distribución por canales de aristas paralelas para reducir solapamientos;
7. colocación de etiquetas en espacios libres próximos a sus elementos;
8. extensión de las *lanes* exteriores; y
9. colocación de objetos de datos y anotaciones.

Esta lógica se reparte entre `sanitize.ts`, `layout-missing.ts`, `pools.ts`, `orthogonal.ts`, `edge-channels.ts`, `labels.ts` y `artifacts.ts`. La división no responde solo a una cuestión de orden interno: permite ajustar una heurística concreta sin tener que reescribir el resto del renderizador.

Robustez ante eventos de frontera sin continuación. Un ejemplo de este tipo de problemas apareció con los eventos de frontera sin continuación en la traza. En ese caso, `bpmn-auto-layout` intentaba procesar una lista de salidas inexistente y el renderizado se detenía por completo.

La corrección no se resolvió como un parche al final del renderizado, sino en el origen del modelo. El AST registra un `boundaryEdgeOverride`, que permite reencaminar correctamente el flujo, y el saneador trata el caso degenerado antes de invocar la librería. El comportamiento quedó cubierto por el test de regresión `tests/render/boundary-completion.test.ts`. Este caso resume bien la estrategia seguida en el proyecto: localizar el fallo, corregir su causa y dejar una prueba que dificulte su reaparición.

4.6 Validación y diagnósticos

Tras el renderizado, el sistema vuelve a examinar el resultado. El módulo `validation/bpmnValidation.ts` ejecuta comprobaciones estructurales sobre el modelo y sobre el *layout*. Cada hallazgo queda clasificado por severidad, `error`, `warning` o `info`, y también por origen, `model` o `layout`.

Los diagnósticos de modelo incluyen:

- `IMPLICIT_START_EVENT`, `IMPLICIT_END_EVENT` e `IMPLICIT_BOUNDARY_END_EVENT`;
- `MISSING_EXPLICIT_START_EVENT` y `MISSING_EXPLICIT_END_EVENT`;
- errores de referencias y extremos en flujos de secuencia y mensaje;
- `MESSAGE_FLOW_WITHIN_POOL` y `POOL_BOUNDARY_CONTROL_FLOW`; y
- incidencias del mapa explícito, como `POOL_MAP_DUPLICATE_LANE`, `POOL_MAP_LANE_UNASSIGNED`, `POOL_MAP_NAME_COLLISION` y `POOL_MAP_UNKNOWN_LANE`.

En el plano gráfico, los diagnósticos incluyen `LAYOUT_ARTIFACT_TOO_FAR`, `LAYOUT_ELEMENT_OVERLAP`, `LAYOUT_UNREADABLE_LABEL` y `LAYOUT_XML_PARSE_ERROR`. Además, la validación recopila medidas de control, como el número de actividades, *gateways*, eventos, nodos aislados y nodos sin salida.

4.7 Desarrollo por hitos y diagramas *canon*

El desarrollo del motor se organizó en hitos incrementales: M1, M2, M3, M4, M4.1, M4.2 y evoluciones posteriores. Esta secuencia quedó reflejada tanto en los cambios del código como en los nombres de las pruebas. Las primeras etapas se centraron en el análisis del DSL; después se incorporó la emisión BPMN; finalmente, el trabajo se desplazó hacia el *layout* y sus refinamientos sucesivos.

Junto a esos hitos se mantuvo un conjunto de diagramas *canon*, usados como regresiones visuales y funcionales. Incluyen procesos elegidos para cubrir situaciones variadas, como la preparación de una hamburguesa, una cinta transportadora, el compostaje, una boda o la llegada de un vuelo. Después de modificar el motor, estos diagramas se regeneran y se revisan propiedades concretas: el sentido de entrada y salida de las flechas, la ausencia de cruces sobre actividades o la separación adecuada de bifurcaciones, uniones y *lanes*.

La combinación de hitos y diagramas *canon* permitió trabajar con un criterio de calidad más estable. Los hitos marcan qué capacidades se añaden; los *canon*, qué nivel visual y funcional debe seguir cumpliéndose. De este modo, una pregunta difícil de formalizar, si el diagrama está bien dibujado, se traduce en comprobaciones más concretas y repetibles.

4.8 Evolución del *system prompt* y mapeo *pool-lane*

El *system prompt* se diseñó íntegramente dentro de este TFM. La sintaxis textual de BPMN Sketch Miner [9] sirvió como punto de partida, pero cada versión incorporó ajustes propios derivados de la experimentación y de la validación con expertos. La evolución fue desde v3.1 hasta v5, con las versiones intermedias v4 y v4.1. En ese recorrido se añadieron recomendaciones de legibilidad, criterios para normalizar eventos de fin y, en V5, un mecanismo explícito de mapeo entre *pools* y *lanes*. Por ello, estas versiones deben entenderse como una aportación original del trabajo, no como la simple reutilización de un *prompt* preexistente.

El bloque final `== pools ==` se introdujo para declarar la agrupación de participantes y sus *swimlanes* sin depender solo de inferencias basadas en la conectividad. Su diseño se especifica en la versión 5 del *system prompt* ([apps/tfm-lab/prompts/system/bpmn_sketch_miner_system_prompt_v5.md](https://github.com/tfm-lab/prompts/system/bpmn_sketch_miner_system_prompt_v5.md)) y se representa en el AST mediante el tipo `DeclaredPool`. Con ello se aborda de manera más controlada una cuestión semántica especialmente sensible: diferenciar participantes independientes y representar su comunicación mediante flujos de mensaje.

Conviene separar esta evolución técnica del protocolo experimental principal. En la matriz de evaluación se mantiene deliberadamente el *system prompt* en v3.1 para aislar el efecto del modelo. Las versiones v4–v5 forman parte del desarrollo del sistema, pero su comparación sistemática se plantea como una ablación y no como base de las conclusiones obtenidas con la matriz v3.1.

4.9 Uso revisado de IA generativa y validación de extremo a extremo

Durante el desarrollo, la IA generativa se empleó como apoyo a la programación, no como sustituto de la revisión técnica. Las propuestas se leyeron, se comprendieron y se ajustaron antes de incorporarse. Además, cada avance se acompañó de observaciones y pruebas concretas, y la validación se realizó sobre el diagrama final, no solo sobre piezas aisladas del código.

Este modo de trabajo fue especialmente importante en el *layout*. En esa parte del motor, una heurística que mejora un caso puede empeorar otro, por lo que los cambios se contrastaron de extremo a extremo con los diagramas *canon*. Cuando aparecía un fallo, se convertía en una prueba de regresión. La IA ayudó a acelerar la escritura de código, pero la fiabilidad del proceso dependió de la división por hitos, la revisión humana y la comprobación sistemática de los resultados.

4.10 Aplicaciones construidas sobre el motor

4.10.1 apps/tfm-lab: laboratorio de investigación

apps/tfm-lab es la herramienta experimental del TFM. Reúne un editor que permite ver en tiempo real el DSL y el diagrama, una pestaña dedicada a los experimentos y un modo *headless* para renderizar lotes de forma automática.

También sirve como espacio de almacenamiento para los *prompts*, los procesos sintéticos SYN001–SYN015 y los resultados de cada ejecución. Las carpetas siguen una nomenclatura trazable e incluyen la entrada, la salida cruda y normalizada, los artefactos BPMN, SVG y PNG, los metadatos, los diagnósticos y los *hashes* de entrada y salida.

4.10.2 apps/company-web: aplicación probada en beta

apps/company-web, en cambio, se orienta al uso por parte de usuarios reales. La aplicación ofrece un flujo guiado para describir el proceso en lenguaje natural, generar el DSL, editar el diagrama y documentarlo. La función **Documentar** exporta el proceso a un documento Word preparado para su distribución. En el repositorio se incluye un documento de ejemplo generado por esta función: [docs/New_Hire_Onboarding_Process.docx](#).

La herramienta se puso a disposición de compañeros de SGS Productivity, que la probaron en fase beta con procesos reales. Esa experiencia aportó evidencia de plausibilidad externa y ayudó a identificar ajustes de usabilidad y maquetación que no aparecían en el corpus sintético.

Por razones técnicas relacionadas con la marca y la licencia, la aplicación presenta la notación como un *DSL textual interno de BPMN*. También evita usar marcas de terceros en los

textos enviados a los modelos o publicados por la herramienta.

4.11 Validación temprana con usuarios reales y colaboración con Metadev

Además de las pruebas internas, el desarrollo contó con dos formas de validación externa que aportan contexto metodológico al trabajo.

Prueba beta con usuarios reales. La aplicación `apps/company-web` se puso a disposición de un grupo de compañeros de SGS Productivity, vinculados a consultoría *Lean* y mejora de procesos, que la utilizaron en fase beta para modelar procesos reales de su actividad. Estos casos eran distintos del corpus sintético, diseñado para controlar la dificultad y las características esperadas: contenían descripciones espontáneas, terminología empresarial, ambigüedades y estructuras propias de procesos en producción. La experiencia tuvo una doble utilidad. Por un lado, ofreció evidencia de plausibilidad externa, al mostrar que personas no expertas en notaciones formales podían obtener diagramas útiles a partir de descripciones en lenguaje natural. Por otro, permitió detectar y corregir ajustes de usabilidad y de maquetación que no se manifestaban en el corpus sintético, como el tratamiento de etiquetas largas, la disposición de carriles o la legibilidad de procesos con múltiples participantes.

Colaboración experta con Metadev. El trabajo contó durante todo el ciclo de desarrollo con el soporte técnico y la mentoría de la empresa tecnológica Metadev. Esta colaboración no fue un acompañamiento puntual, sino una supervisión continuada por parte de expertos del sector. Su papel fue especialmente relevante en la especificación, el diseño y la validación de la gramática del DSL de representación intermedia. Contar con ese criterio experto durante el diseño del lenguaje y de su *system prompt* reforzó la solidez de las decisiones de modelado y añadió una dimensión de validación que un desarrollo puramente académico no habría cubierto del mismo modo.

4.12 Pipeline de evaluación y suite de pruebas

El *pipeline* de evaluación se encuentra en el directorio `TFM-eval/` y está implementado en Python. Su función es procesar los experimentos, renderizarlos mediante el motor compartido y calcular métricas sin depender de un diagrama de referencia. Entre esas métricas se incluyen la validez conforme a XML Schema Definition (XSD), la conectividad, el grado de los elementos, la completitud de decisiones y etiquetas, el emparejamiento de *gateways*, el tamaño, la complejidad del flujo de control y la ciclicidad.

El paquete incorpora los esquemas XSD de BPMN 2.0 y ofrece una interfaz de línea de comandos en `TFM-eval/src/bpmn_eval/cli.py` y `TFM-eval/src/bpmn_eval/cli_experiments.py`. A partir de ella se generan `results.csv` y `results.json`. Con esos resultados como entrada, el análisis y las figuras se elaboran en un cuaderno independiente (`TFM-eval/notebooks/analysis.ipynb`). La cadena completa de renderizado y evaluación puede ejecutarse mediante `scripts/run-evaluation.ps1`, utilizando Playwright y Chromium en modo *headless*.

La verificabilidad del sistema se apoya, además, en una suite de pruebas que cubre:

- el *parser* y el análisis semántico del DSL;
- la emisión BPMN y el mapeo de *pools*;
- el enrutado ortogonal, los eventos de frontera, los artefactos y el *layout* de participantes;
- la validación estructural y los diagnósticos; y
- los diagramas *canon* como regresiones de extremo a extremo.

4.13 Síntesis

La arquitectura elegida responde a la doble naturaleza del trabajo. Por un lado, sostiene una investigación reproducible, formada por el motor, el laboratorio y el *pipeline* de evaluación. Por otro, permite disponer de una herramienta aplicada, probada en un contexto profesional. En ambos casos se utiliza el mismo motor, lo que evita separar artificialmente la parte experimental de la parte demostrativa.

El desarrollo incremental, los diagnósticos estables, la separación entre semántica y representación gráfica y los diagramas *canon* hacen posible abordar un problema visual y complejo con un sistema cuya calidad puede comprobarse de forma automática.

Evaluación

5.1 Objetivo de la evaluación

La evaluación estudia la calidad de los diagramas BPMN 2.0 generados por distintos modelos de lenguaje a partir de descripciones textuales de procesos. El análisis se plantea de forma automática y reproducible, y no exige disponer de un diagrama BPMN de referencia para cada caso.

La aplicación evaluada sigue una arquitectura en dos etapas (véase el diagrama general del *pipeline* en la Figura 4.1). Primero, el modelo de lenguaje no produce XML BPMN directamente, sino una representación intermedia en un lenguaje de dominio específico. La sintaxis de este lenguaje toma como referencia la documentada por BPMN Sketch Miner [9]. Después, un compilador determinista transforma ese DSL en un diagrama BPMN 2.0 renderizable. Esta separación es importante porque permite distinguir los fallos atribuibles al modelo, como procesos incompletos, mal estructurados o semánticamente insuficientes, de los problemas que pertenecen a la compilación o al renderizado.

El diseño de la evaluación toma como punto de partida BEF4LLM, propuesto por Lauer et al. [3]. Este marco organiza la evaluación de modelos BPMN generados por LLM en cuatro dimensiones: calidad sintáctica, calidad pragmática, calidad semántica y validez del fichero BPMN-XML. Aunque BEF4LLM ya se introdujo de forma resumida en los capítulos anteriores, este capítulo desarrolla con más detalle su adaptación al presente trabajo (Sección 5.3.1). La adaptación es necesaria por dos razones. Por un lado, BEF4LLM incluye 39 métricas, y varias de ellas asumen que existe un BPMN de referencia. Por otro, en este TFM el XML final no sale directamente del modelo, sino de un compilador determinista. Por ello, se emplea un subconjunto operativo de métricas que puede calcularse de forma fiable a partir de los artefactos realmente disponibles.

También se considera el sistema 3C, presentado en la introducción como antecedente para valorar Claridad, Corrección y Completitud [8]. No se adopta como instrumento principal porque requiere valoración ordinal por evaluadores humanos o una aproximación de tipo *LLM-as-a-judge*. Ese enfoque sería útil para estudiar la usabilidad del resultado, pero introduciría subjetividad, dependencia del evaluador y menor reproducibilidad. Dado que este capítulo se

centra en evaluar automáticamente el comportamiento del *pipeline*, las dimensiones de 3C se trasladan a indicadores observables: la robustez y la validez aproximan la corrección; la cobertura de características esperadas aproxima la completitud; y los diagnósticos estructurales ayudan a explicar defectos concretos. La claridad visual queda fuera del núcleo cuantitativo, ya que depende del *layout* y de la interpretación humana del diagrama.

La Tabla 5.1 recoge las preguntas de investigación que guían la evaluación:

Código	Pregunta de investigación	Enfoque de evaluación
RQ1	¿El enfoque basado en DSL, genera BPMN 2.0 válido y renderizable?	Robustez sintáctica, semántica y de renderizado.
RQ2	¿Qué modelo produce diagramas de mayor calidad con el mismo prompt?	Comparación entre modelos con métricas estructurales y semánticas.
RQ3	¿Cómo evoluciona la calidad al aumentar la dificultad del proceso?	Comparación por niveles de dificultad: fácil, medio, difícil y estrés.
RQ4	¿Los diagramas capturan las características semánticas esperadas?	Cobertura de <i>features</i> esperadas y análisis de códigos diagnósticos.
RQ5	¿Cuánto influye el <i>prompt</i> de sistema en la calidad final?	Ablación entre versiones del <i>prompt</i> manteniendo proceso y modelo.
RQ6	¿Cómo afecta la longitud o detalle de la descripción de entrada?	Discusión razonada a partir del coste en <i>tokens</i> y la cobertura por versión; evaluación controlada reservada a trabajo futuro.
RQ7	¿Los modelos son estables entre ejecuciones repetidas?	Variabilidad intra-modelo e intra-proceso entre R01, R02 y R03.
RQ8	¿Cómo se compara el enfoque DSL con la generación directa <i>zero-shot</i> ?	Comparación controlada frente a BPMN-XML directo; imagen y MCP quedan como extensiones futuras.

Tabla 5.1: Preguntas de investigación de la evaluación

5.2 Diseño experimental

La evaluación se apoya en un conjunto controlado de procesos sintéticos, identificados como SYN001–SYN015. Estos procesos se diseñaron para cubrir distintos niveles de dificultad y varias construcciones BPMN. Cada modelo ejecuta varias veces cada proceso, lo que permite analizar no solo la calidad media, sino también la estabilidad entre ejecuciones.

El corpus se construyó expresamente para este trabajo. Su objetivo es cubrir de forma gradual las construcciones BPMN más relevantes: secuencias, compuertas exclusivas y paralelas, eventos de inicio, fin, temporizador y excepción, objetos de datos, *lanes* y colaboración entre *pools* con flujos de mensaje. Cada proceso incluye una etiqueta de dificultad y una lista de características esperadas, utilizada después para calcular la cobertura semántica. La composición detallada por grupos de dificultad se recoge en el Apéndice (Tabla A.6). Como referencia, un caso de dificultad media describe en lenguaje natural un proceso de aprobación, por ejemplo, la tramitación de una solicitud, con una decisión (compuerta XOR con ramas de aprobación

y rechazo), varias tareas asignadas a roles distintos y un evento de fin. Un caso de estrés, en cambio, reúne en un único diagrama varios participantes en *pools* independientes, flujos de mensaje, ramas paralelas y excepciones.

El corpus principal se organiza en cuatro niveles de dificultad (Tabla 5.2):

Nivel	Descripción	Finalidad
Fácil	Procesos lineales o con ramificaciones simples.	Comprobar si el sistema resuelve correctamente los casos básicos.
Medio	Procesos con decisiones, paralelismos, datos o lanes.	Evaluar estructuras frecuentes en procesos empresariales reales.
Difícil	Procesos con eventos, excepciones o colaboración más compleja.	Medir la capacidad de modelar situaciones BPMN avanzadas.
Estrés	Procesos densos o con múltiples requisitos simultáneos.	Identificar los límites prácticos del enfoque.

Tabla 5.2: Niveles de dificultad del corpus sintético

Los modelos con matriz completa de experimentos para el *prompt* v3.1 son GPT_5_5_THINKING, OPUS_4_7 y GEMINI_3_5_FLASH_THINKING. Para cada uno se realizaron 45 ejecuciones, correspondientes a 15 procesos con 3 repeticiones. Por tanto, el ranking principal se basa en 135 observaciones equilibradas. El fichero consolidado incluye también ablaciones de *prompt*, modelos exploratorios y 45 ejecuciones de generación BPMN-XML directa. Estas observaciones se analizan en las preguntas de investigación correspondientes, pero se mantienen separadas del ranking principal.

La evaluación se implementa como un *pipeline* reproducible dividido en dos fases:

1. **Fase A: generación, renderizado y enriquecimiento de resultados.** Comprende la ejecución *headless*, la compilación del DSL, el renderizado del BPMN, el cálculo de las métricas y la escritura de las salidas agregadas en `results.csv` y `results.json`.
2. **Fase B: análisis y visualización.** Un *notebook* utiliza exclusivamente los resultados producidos en la fase anterior para generar las tablas y figuras del capítulo.

Una decisión importante es que las métricas no se calculan directamente en el *notebook*. Todas están implementadas y probadas dentro del paquete de evaluación. Así se evita que el análisis dependa de código *ad hoc* o difícil de reproducir.

5.3 Métricas utilizadas

La evaluación combina métricas de robustez, estructura, semántica y estabilidad.

5.3.1 Relación con BEF4LLM

BEF4LLM ofrece un marco amplio para evaluar BPMN generados por LLM y agrupa los resultados por dimensiones de calidad [3]. Distingue una medida de validez BPMN-XML, 16 métricas de calidad sintáctica, 15 de calidad pragmática y 7 de calidad semántica. La validez se comprueba frente al esquema XSD de BPMN 2.0. La calidad sintáctica incluye, entre otros aspectos, la presencia de eventos de inicio y fin, las reglas de conexión de flujos, el etiquetado de tareas y los grados de entrada y salida de eventos, tareas y compuertas. La calidad pragmática se aproxima a la legibilidad mediante el tamaño, la densidad, la conectividad, la particionabilidad, la concurrencia y otras propiedades de complejidad. Por último, la calidad semántica compara el modelo candidato con uno de referencia a partir de la similitud de etiquetas, la estructura del grafo y el comportamiento.

No usar directamente las métricas de distancia respecto a modelos humanos no implica descartar BEF4LLM. Supone, más bien, reconocer una limitación relevante para este contexto. En BPMN no siempre existe una única representación correcta: dos diagramas con diferencias estructurales o visuales pueden expresar una semántica de negocio equivalente. También puede ocurrir lo contrario: dos diagramas parecidos en sus etiquetas o en su estructura superficial pueden diferir en aspectos semánticos importantes, como la separación entre participantes o el uso de flujos de mensaje. Una métrica basada en la distancia a un diagrama humano de referencia podría penalizar alternativas válidas o sobrevalorar diagramas formalmente similares pero incompletos. Por esta razón, el TFM evalúa si el diagrama generado conserva las propiedades BPMN exigidas por el enunciado y si mantiene consistencia estructural.

Este TFM adopta el planteamiento objetivo y multidimensional de BEF4LLM, aunque no incorpora todas sus métricas. Los motivos son los siguientes:

1. **Ausencia de un BPMN de referencia por caso.** Las métricas semánticas de BEF4LLM comparan el diagrama candidato con otro de referencia. Dado que este trabajo no utiliza una referencia manual, dichas métricas se sustituyen por la cobertura de características esperadas.
2. **Multiplicidad de diagramas válidos.** Un mismo proceso admite distintas representaciones BPMN sin que estas tengan necesariamente una calidad inferior. Por ello, la distancia respecto a un único diagrama humano no se considera suficiente para valorar la calidad semántica del resultado.
3. **Arquitectura DSL + compilador.** La validez del XML está muy condicionada por el compilador determinista. En consecuencia, la métrica M1 se presenta como comprobación básica de validez, pero no se emplea para ordenar los modelos cuando alcanza valores de saturación.
4. **Evitar penalizar decisiones del motor.** Algunas reglas estrictas de BPMN-XML dependen de la forma en que el compilador materializa el DSL. Por ejemplo, exigir expresiones de condición en todas las ramas penalizaría una decisión de implementación y no necesariamente un error del LLM.

5. Reproducibilidad y trazabilidad. Se incluyen únicamente las métricas que pueden calcularse de manera automática, estable y auditable sobre las salidas actuales del *pipeline*.

El paquete calcula diez métricas, identificadas como M1–M10. M1–M5 y M8 funcionan como indicadores de calidad o control. M6, M7, M9 y M10, en cambio, describen tamaño y complejidad; por ello no se integran en la media `scored_mean`. La Tabla 5.3 resume el subconjunto operativo de métricas.

Métrica	Interpretación	Uso en este TFM
M1	Validez XSD del BPMN 2.0 generado.	Comprobación básica de validez; no se incluye en <code>scored_mean</code> por saturación.
M2	Conectividad estructural.	Comprueba si los elementos del proceso quedan conectados.
M3	Grado de los elementos.	Penaliza entradas o salidas anómalas en tareas, eventos y compuertas.
M4	Complejidad de decisiones.	Evalúa si las ramas salientes de compuertas XOR están etiquetadas.
M5	Complejidad de etiquetas.	Evalúa si los elementos relevantes tienen nombres informativos.
M6	Tamaño total del modelo (<code>tnn</code>).	Descriptor: número total de nodos del diagrama.
M7	Complejidad del flujo de control (<code>cfc_total</code>).	Descriptor: complejidad introducida por las compuertas y sus ramas.
M8	Emparejamiento de gateways.	Mide si aperturas y cierres de compuertas están balanceados.
M9	Coefficiente de conectividad de red (<code>cnc_value</code>).	Descriptor de la relación entre flujos de secuencia y nodos.
M10	Ciclicidad (<code>has_cycle</code>).	Descriptor binario que indica la presencia de ciclos en el grafo.

Tabla 5.3: Subconjunto operativo de métricas inspirado en BEF4LLM

M6, M7, M9 y M10 no deben interpretarse como puntuaciones de calidad acotadas en $[0, 1]$. Son magnitudes descriptivas sobre el tamaño y la complejidad del grafo. En particular, la complejidad del flujo de control (M7, `cfc_total`) sigue la métrica *Control-Flow Complexity* de Cardoso [38]. Se trata de un número entero y adimensional, calculado como la suma de las contribuciones de las compuertas divergentes: una compuerta XOR con n ramas aporta n ; una compuerta paralela (AND), 1; y una inclusiva (OR) con n ramas, $2^n - 1$. Un valor alto de `cfc_total` indica que quien interpreta el proceso debe contemplar más estados de decisión. Por eso se utiliza como indicador de dificultad estructural, no como medida de corrección.

5.3.2 Robustez

La robustez se resume mediante dos tasas de éxito (Tabla 5.4):

Métrica	Definición	Qué mide
<code>semantic_success</code>	DSL: <code>parserValid</code> $\wedge$ semanticValid; XML di- recto: <code>bpmnXmlValid</code> $\wedge$ <code>bpmnImportValid</code>	Si la representación genera- da puede interpretarse co- mo un modelo BPMN válido.
<code>render_success</code>	<code>renderValid</code>	Si el diagrama pudo rende- rizarse correctamente.

Tabla 5.4: Métricas de robustez del pipeline

La separación entre `semantic_success` y `render_success` evita mezclar dos tipos de fallo. Los errores de generación, atribuibles al modelo, se distinguen de los errores del motor de renderizado. En los experimentos de BPMN-XML directo, las métricas M1–M10 y la cobertura se calculan siempre sobre el fichero crudo devuelto por el modelo. Si el XML no incluye BPMN DI, puede aplicarse `bpmn-auto-layout` solo para visualizarlo; el artefacto maquetado no sustituye al original en la puntuación. Cuando el XML está mal formado, recibe M1 = 0 y puntuaciones estructurales nulas, sin reparaciones manuales.

5.3.3 Calidad estructural

La calidad estructural se resume en el promedio compuesto `scored_mean`, con valores entre 0 y 1. Este promedio incorpora las métricas que discriminan mejor la calidad del diagrama generado: M2, M3, M4, M5 y M8. La validez XSD, correspondiente a M1, se informa como comprobación básica de validez, pero no forma parte del promedio compuesto. M1 se considera *saturada* porque el compilador determinista produce casi siempre un XML conforme al esquema. En la práctica, toma el valor máximo o próximo a 1 en casi todas las ejecuciones. Al variar muy poco entre modelos, apenas discrimina; incluirla en `scored_mean` desplazaría todas las medias hacia arriba sin modificar sustancialmente el orden. Por ese motivo se excluye del promedio compuesto.

Para comparar representaciones se usan dos agregaciones. La **calidad condicional** es la media de `scored_mean` calculada solo sobre las ejecuciones con éxito semántico. La **calidad penalizada**, almacenada como `scored_mean_penalized`, incluye todas las ejecuciones y asigna un valor 0 a los fallos:

$$\overline{Q}_{\text{pen}} = \frac{1}{N} \sum_{i=1}^N \begin{cases} \text{scored_mean}_i, & \text{si la ejecución tiene éxito semántico,} \\ 0, & \text{en caso contrario.} \end{cases}$$

Esta magnitud equivale al producto entre la tasa de éxito semántico y la calidad media condicional. Por ello, la evaluación presenta la robustez y la calidad como dimensiones independientes, y utiliza la calidad penalizada cuando compara enfoques con tasas de éxito dife-

rentes. Así se evita el sesgo de supervivencia que aparecería si se excluyeran del promedio los procesos que una representación no logra generar.

La definición de M4 se adapta respecto a una lectura más estricta de BPMN, basada en expresiones de condición y flujos por defecto. En este compilador, esa lectura penalizaría una decisión de implementación del motor y no necesariamente un error del modelo. Por ello, M4 se interpreta como completitud de decisiones: para cada gateway XOR divergente, se calcula la proporción de ramas salientes con una etiqueta no vacía.

5.3.4 Cobertura semántica

La evaluación semántica se realiza sin diagrama de referencia. En lugar de comparar cada resultado con un modelo BPMN ideal, se define para cada proceso una lista de características esperadas. Entre ellas se incluyen compuertas XOR, compuertas paralelas, lanes, pools, *message flows*, objetos de datos, eventos temporales o eventos de excepción.

Después, un conjunto de detectores automáticos inspecciona el resultado y comprueba si cada característica esperada aparece en el diagrama generado. La cobertura de un caso se calcula como la proporción de características esperadas que se detectan: si un proceso declara E características esperadas y los detectores localizan D de ellas en el diagrama, la cobertura es D/E , un valor entre 0 (no se recoge ninguna) y 1 (se recogen todas). Cada detector aplica una comprobación determinista sobre el modelo BPMN resuelto, por ejemplo, la existencia de al menos una compuerta paralela, de más de un *pool* o de algún *message flow*. La presencia o ausencia de cada característica se decide, por tanto, de forma binaria y reproducible. La cobertura media reportada por modelo o por dificultad es el promedio de estos cocientes sobre los casos correspondientes. La métrica no certifica equivalencia semántica completa, pero permite comprobar si el modelo recoge los elementos BPMN principales exigidos por el enunciado.

La cobertura también se agrupa según `system_prompt_version`. Esta dimensión es necesaria para RQ5. La comparación principal entre modelos se restringe a v3.1, mientras que la ablación contrasta v3.1, v4 y v5 sin fusionar sus resultados bajo una única etiqueta `representation=dsl`. Si una versión del *prompt* no permite expresar ciertas características de colaboración, esas omisiones deben leerse como limitaciones de esa configuración, no necesariamente como fallos del modelo.

5.3.5 Consumo estimado de tokens

El *pipeline* estima fuera de línea los tokens del *system prompt*, de la descripción del proceso y de la salida cruda mediante la codificación fija `o200k_base`. Los tokens de sistema, proceso, entrada total, salida y total acumulado se guardan por separado. Esta medición permite comparar la extensión textual de las representaciones, pero no equivale a la facturación real de ningún proveedor: excluye tokens de razonamiento ocultos, envoltorios de conversación, diferencias de tokenización, reglas de caché y precios específicos de cada API.

5.3.6 Diagnósticos del motor

Además de las métricas numéricas, se recopilan los códigos diagnósticos emitidos por el motor. Estos códigos clasifican errores y normalizaciones: eventos de inicio o fin implícitos, ramas que no alcanzan un evento final, referencias inválidas en *sequence flows*, errores de anclaje de fragmentos, objetos de datos duplicados o completado automático de eventos de borde sin salida.

Su valor está en que permiten localizar defectos estructurales concretos incluso cuando la puntuación global parece alta.

5.4 Visión general del dataset

El conjunto consolidado reúne 420 experimentos trazables: 177 corresponden al DSL con *system prompt* v3.1, 99 a v4, 99 a v5 y 45 a la generación BPMN-XML directa *zero-shot*. La comparación principal entre modelos se limita a las 135 ejecuciones equilibradas de v3.1. La Tabla 5.5 muestra la distribución de estados del subconjunto DSL v3.1:

Estado	Casos	Interpretación
success_with_warnings	151	El diagrama compila, aunque el motor aplica normalizaciones o emite advertencias.
success	15	El diagrama compila sin advertencias.
semantic_error	11	El DSL no supera la validación semántica.
render_error	0	No se observan errores de renderizado en el conjunto final.

Tabla 5.5: Distribución de estados de compilación del subconjunto DSL v3.1

La presencia de advertencias no implica por sí sola que el diagrama sea incorrecto. En muchos casos responden a normalizaciones benignas, como la creación implícita de eventos de inicio o fin cuando el DSL generado no los declara de forma expresa.

De las 177 observaciones de v3.1, 135 forman la matriz equilibrada de tres modelos, quince procesos y tres repeticiones. Las 42 restantes tienen carácter exploratorio: 24 de OPUS_4_8_MEDIUM, 12 de GPT_5_4_THINKING, 5 de 3_1_PRO y 1 de OPUS_ADAPTATIVE. Las matrices v4 y v5 se reservan para RQ5, mientras que las 45 salidas ZSXML se estudian en RQ8. Mantener esta separación evita mezclar versiones de *prompt* o representaciones distintas en una misma media agregada.

La Tabla 5.6 muestra los modelos con cobertura completa y, por tanto, comparables de forma directa:

El resto de modelos se mantiene como resultado exploratorio, ya que no cubre el mismo *grid* en todas las configuraciones. Opus 4.8 Medium, por ejemplo, combina 24 ejecuciones de v3.1 con dos subconjuntos de 9 ejecuciones de v4 y v5, centrados en procesos con distinta

Modelo	Ejec.	Uso en el análisis
GPT_5_5_THINKING	45	Modelo completo, incluido en el ranking principal.
OPUS_4_7	45	Modelo completo, incluido en el ranking principal.
GEMINI_3_5_FLASH_THINKING	45	Modelo completo, incluido en el ranking principal.

Tabla 5.6: Modelos con cobertura experimental completa

dificultad. Las medias de estas versiones no son comparables; por ello se excluyen de los rankings hasta disponer de una matriz equilibrada.

5.5 RQ1: Robustez del enfoque DSL

La primera pregunta de investigación se centra en la robustez básica del enfoque: si el *pipeline* basado en DSL produce de forma sistemática diagramas BPMN válidos y renderizables.

Modelo	n	Éxito semántico	Éxito renderizado
GPT_5_5_THINKING	45	100 %	100 %
OPUS_4_7	45	96 %	96 %
GEMINI_3_5_FLASH_THINKING	45	96 %	96 %

Tabla 5.7: Tasa de éxito semántico y de renderizado (%) por modelo. Las tres configuraciones superan el 95 %, sin diferencias relevantes en robustez.

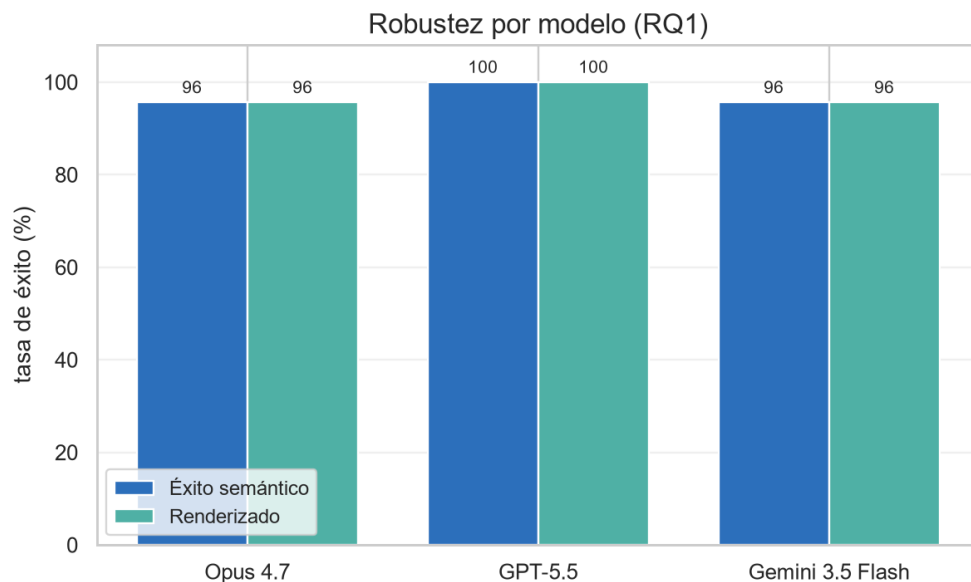


Figura 5.1: Robustez por modelo

Los datos (Tabla 5.7 y Figura 5.1) muestran una robustez elevada en los tres modelos principales. GPT_5_5_THINKING completa todos los experimentos sin errores semánticos ni de renderizado. OPUS_4_7 y GEMINI_3_5_FLASH_THINKING alcanzan el 96 % de éxito, de modo que los fallos restantes son puntuales y se concentran en casos de mayor complejidad.

En el conjunto final, la tasa de renderizado coincide con la de éxito semántico. Esto sugiere que los fallos restantes no proceden del *renderer*, sino de ejecuciones en las que el modelo no llegó a generar un DSL semánticamente válido.

5.6 RQ2: Comparación de calidad entre modelos

La segunda pregunta compara la calidad media de los diagramas generados por los modelos con matriz completa. Para que la comparación sea limpia, se mantienen constantes el *prompt* y el conjunto de procesos.

Modelo	n	scored_mean	M1	M2	M3	M4	M5	M8
GPT_5_5_THINKING	45	0,875 ± 0,039	1,00	0,93	1,00	0,86	0,90	0,70
OPUS_4_7	45	0,833 ± 0,065	0,96	0,92	0,95	0,90	0,80	0,59
GEMINI_3_5_FLASH_THINKING	45	0,804 ± 0,067	0,96	0,92	0,95	0,82	0,81	0,51

Tabla 5.8: Calidad estructural media por modelo

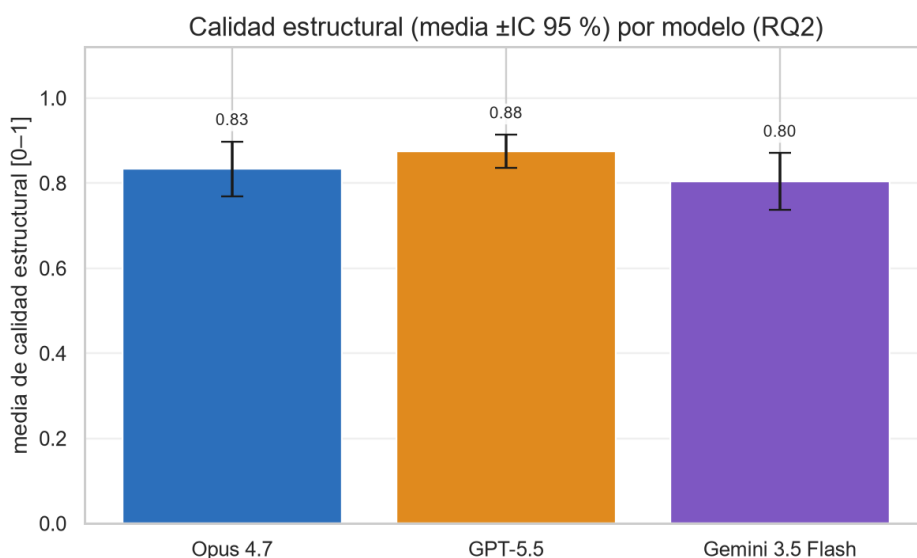


Figura 5.2: Media de calidad estructural por modelo

La media compuesta más alta corresponde a GPT_5_5_THINKING, seguido de OPUS_4_7 y GEMINI_3_5_FLASH_THINKING (Tabla 5.8 y Figura 5.2). En la Tabla 5.8, el valor situado tras el signo $\pm$ es la desviación típica de *scored_mean* sobre las 45 ejecuciones de cada modelo. Mide la dispersión de la calidad entre ejecuciones y no debe interpretarse como un error de medida. Aun así, las diferencias deben leerse con prudencia porque los intervalos de confianza se solapan. El orden descriptivo no basta, por sí solo, para afirmar que exista una diferencia estadísticamente significativa.

M1, M2 y M3 presentan valores próximos a la saturación. Una vez que el modelo produce un DSL válido, el compilador suele generar diagramas BPMN bien conectados y formalmente consistentes. Las diferencias aparecen sobre todo en M4 y M8. M4 refleja tanto la creación de una

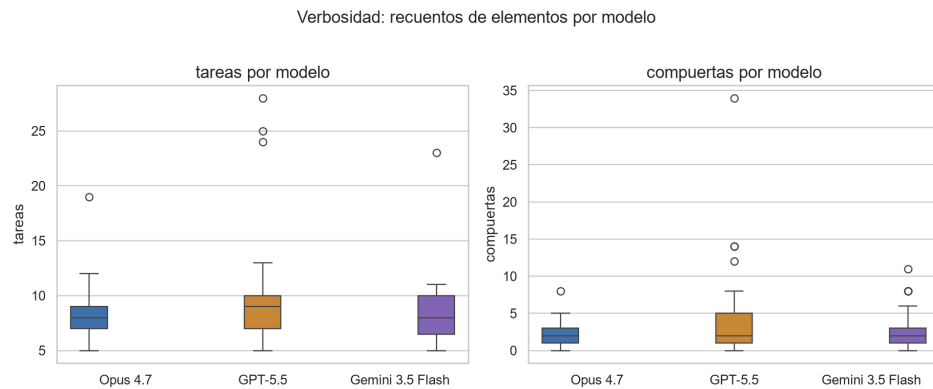


Figura 5.3: Distribución del número de tareas y de compuertas por modelo. Los tres modelos presentan una verbosidad similar, con algún valor atípico en procesos complejos.

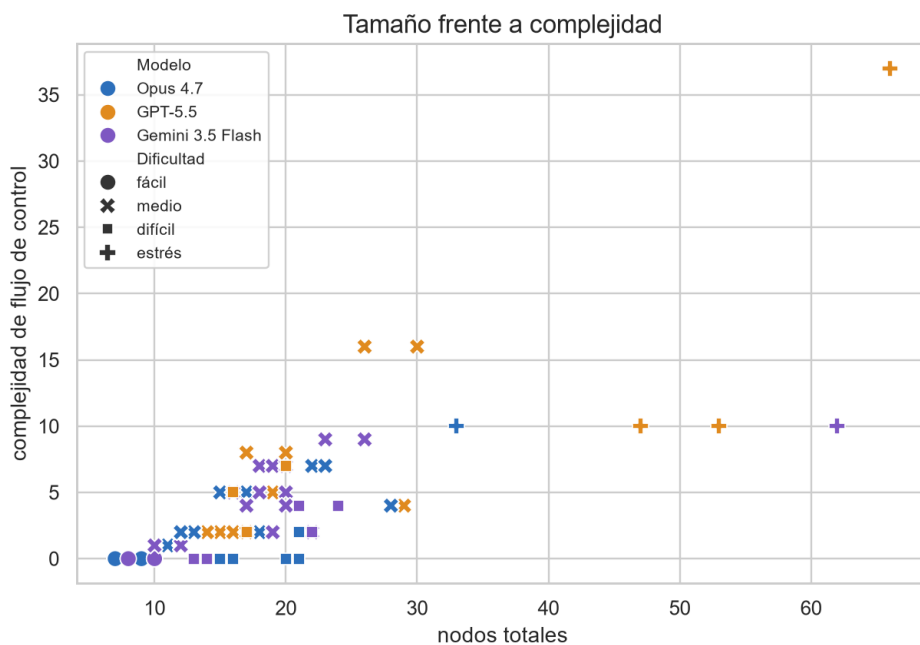


Figura 5.4: Relación entre el número total de nodos y la complejidad de flujo de control de los diagramas generados, por modelo y nivel de dificultad.

compuerta como la explicitación de la lógica de decisión. M8, por su parte, mide si las estructuras de ramificación y sincronización quedan correctamente balanceadas, algo especialmente relevante en procesos con paralelismos, excepciones o flujos alternativos.

La distribución del recuento de elementos por modelo (Figura 5.3) y la relación entre tamaño y complejidad (Figura 5.4) apuntan en la misma dirección: los tres modelos tienen perfiles de verbosidad y complejidad parecidos, sin diferencias acusadas en el número de tareas o compuertas.

En conjunto, los tres modelos completos producen resultados sólidos. La diferencia principal no está solo en la media agregada, sino en el tipo de errores que cometen. Por ello, la elección del modelo debería considerar también qué procesos se quieren modelar y qué fallos serían más costosos en el uso previsto.

5.7 RQ3: Efecto de la dificultad del proceso

La tercera pregunta analiza cómo cambia la calidad cuando aumenta la dificultad del proceso descrito.

Dificultad	n	Éxito sem.	Éxito rend.	scored_mean	Cobertura
Fácil	27	100 %	100 %	0,984	1,00
Media	81	100 %	100 %	0,841	0,84
Difícil	18	100 %	100 %	0,788	0,76
Estrés	9	56 %	56 %	0,463	0,46

Tabla 5.9: Efecto de la dificultad del proceso

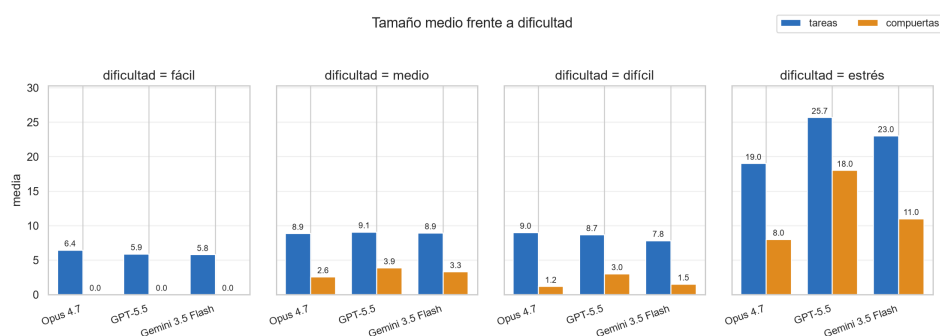


Figura 5.5: Número medio de tareas y de compuertas por modelo y nivel de dificultad. El tamaño del diagrama crece de forma marcada en el nivel de estrés.

La dificultad del proceso es el factor con el efecto más visible (Tabla 5.9). La calidad disminuye de forma monótonica a medida que crece la complejidad: los casos fáciles alcanzan una media prácticamente perfecta, los medios y difíciles mantienen una calidad razonablemente alta, y los de estrés muestran una caída marcada.

El enfoque DSL funciona muy bien en procesos sencillos y medios, y sigue siendo útil en los difíciles. Sus límites aparecen cuando se acumulan muchas exigencias BPMN al mismo tiempo.

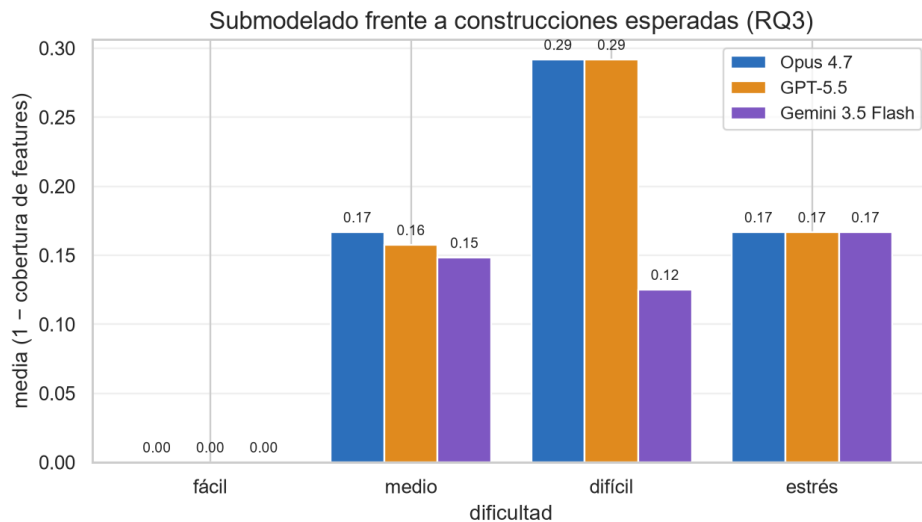


Figura 5.6: Submodelado en procesos complejos

Los procesos de estrés combinan colaboración entre participantes, excepciones, varias ramas, eventos y requisitos de modelado que deben mantenerse coherentes de forma simultánea.

El aumento del tamaño del diagrama con la dificultad (Figura 5.5) y el submodelado observado en los procesos más complejos (Figura 5.6) ilustran gráficamente este patrón.

En el conjunto final no se observan fallos de renderizado en los casos difíciles. Por tanto, la reducción de calidad se atribuye a la complejidad del proceso descrito, no a un problema del motor de visualización.

5.8 RQ4: Adecuación semántica

La cuarta pregunta estudia si los diagramas generados incorporan las características BPMN que exige el enunciado de cada proceso.

5.8.1 Cobertura de características esperadas

La Tabla 5.10 recoge la cobertura media de *features* en los tres modelos completos:

Modelo	Cobertura media
GPT_5_5_THINKING	0,86
GEMINI_3_5_FLASH_THINKING	0,85
OPUS_4_7	0,81

Tabla 5.10: Cobertura media de características esperadas

Las *features* omitidas con mayor frecuencia en el conjunto completo se recogen en la Tabla 5.11:

Feature omitida	Omisiones	Interpretación
message_flow	27	El modelo no representa correctamente comunicaciones entre participantes.
multiple_pools	18	El proceso se colapsa en un único pool cuando deberían existir varios participantes.
send_receive	18	Se omiten eventos o tareas explícitas de envío o recepción.
cancellation / exception_flow	5 / 5	Las excepciones y cancelaciones se modelan de forma incompleta.
annotations, multiple_lanes, xor_gateway, parallel_gateway, data_objects, timer_or_exception_event	4 cada una	Omisiones dispersas en procesos de mayor dificultad.

Tabla 5.11: Características esperadas omitidas con mayor frecuencia

La principal debilidad semántica aparece en la colaboración entre participantes. Los procesos que requieren varios pools y flujos de mensaje son los que con más frecuencia se simplifican en un único pool. Cuando ocurre esto, se pierde la separación entre actores y la comunicación explícita mediante `messageFlow`.

Esta distinción importa porque, en BPMN, la diferencia entre *sequence flow* y *message flow* no es meramente visual. Un *sequence flow* expresa continuidad dentro de un mismo proceso o participante; un *message flow* representa interacción entre participantes independientes. Si el modelo agrupa varios participantes en un único pool, el diagrama puede seguir siendo comprensible, pero pierde fidelidad semántica respecto al proceso real.

La debilidad en el modelado de múltiples *pools* se explica, en buena medida, por una limitación heredada de la sintaxis tomada como punto de partida. BPMN Sketch Miner [9] no implementaba los *pools* de forma completamente explícita en el DSL. El agrupamiento de carriles (*lanes*) dentro de *pools* se realizaba de forma automática e implícita a partir de la conectividad del flujo de control. Al no existir un control directo de estas fronteras, cuando el LLM omitía la conexión de los flujos de mensaje, los participantes independientes podían quedar integrados erróneamente en un mismo pool. Como se detalla en la Sección 5.9, la versión 5 del *system prompt* corrige esta limitación al incorporar el bloque de declaración explícita `== pools ==`, que permite delimitar y nombrar las fronteras organizativas directamente en el DSL.

5.8.2 Taxonomía de diagnósticos

Los diagnósticos del motor complementan la puntuación numérica con una clasificación de problemas concretos (Tabla 5.12).

Los diagnósticos más frecuentes están relacionados con normalizaciones de inicio y fin. No implican necesariamente errores graves del modelo. Los códigos menos habituales, en cambio, son más informativos para detectar defectos estructurales reales.

Código diagnóstico	OPUS	GPT	GEMINI	Interpretación
IMPLICIT_END_EVENT	104	51	93	Normalización frecuente: el motor añade finales implícitos.
IMPLICIT_START_EVENT	33	29	39	Normalización frecuente: el motor añade inicios implícitos.
MISSING_EXPLICIT_END_EVENT	34	20	36	El DSL omite finales explícitos.
MISSING_EXPLICIT_START_EVENT	23	25	15	El DSL omite inicios explícitos.
DUPLICATED_DATA_OBJECT	3	1	3	Se repiten objetos de datos.
AP-7	3	0	3	Violación de regla de anclaje de fragmentos.
BRANCH_WITHOUT_END_REACHABILITY	0	3	0	Ramas que no alcanzan un evento final.
ACTIVITY_WITHOUT_INCOMING	1	0	1	Actividad sin flujo entrante.
SEQUENCE_FLOW_INVALID_REFERENCE	1	0	1	Referencia inválida en un flujo de secuencia.
IMPLICIT_BOUNDARY_END_EVENT	4	3	5	Evento de borde sin salida completado por el motor.

Tabla 5.12: Taxonomía de diagnósticos por modelo

OPUS_4_7 y GEMINI_3_5_FLASH_THINKING registran algunos casos de AP-7, vinculados a problemas de anclaje de fragmentos. GPT_5_5_THINKING no presenta ese error, aunque sí algunos casos de ramas que no alcanzan un evento final. Esto muestra que, incluso con medias agregadas similares, los modelos pueden fallar de maneras distintas.

5.9 RQ5: Sensibilidad al prompt de sistema

La quinta pregunta analiza cuánto depende la cobertura semántica de la versión del *prompt* de sistema. Para no mezclar este efecto con la comparación entre modelos de RQ2, el ranking principal se mantiene restringido a v3.1. La ablación de RQ5 agrupa explícitamente los resultados por *system_prompt_version* y compara matrices completas de 45 ejecuciones por versión para GPT-5.5 y Gemini 3.5.

Modelo	v3.1	v4	v5
GPT-5.5	0,856	0,828	0,957
Gemini 3.5	0,846	0,830	0,911

Tabla 5.13: Cobertura media de características por versión del *system prompt*

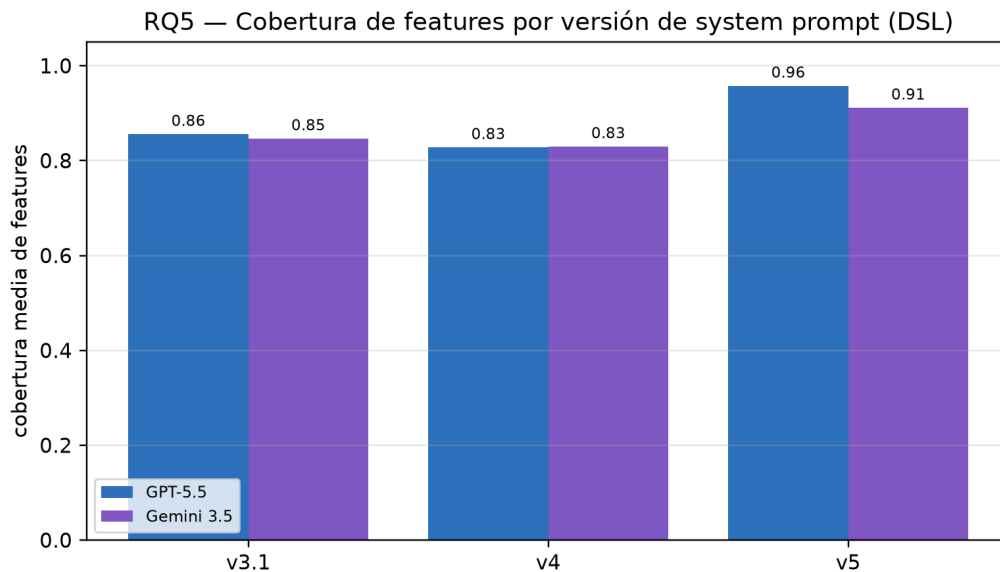


Figura 5.7: Ablación de cobertura semántica por versión del *system prompt*

La versión v4 no mejora la cobertura de v3.1. V5, en cambio, la eleva hasta 0,957 en GPT-5.5 y 0,911 en Gemini 3.5 (Tabla 5.13 y Figura 5.7). La mejora se concentra en las características de colaboración, `multiple_pools`, `message_flow` y `send_receive`, cuya expresión estaba limitada por las instrucciones de v3.1 y v4. Por tanto, una parte importante de la debilidad observada en RQ4 no apunta a una incapacidad intrínseca del DSL o del modelo, sino a la configuración concreta del *prompt* controlado.

Con v5, GPT-5.5 alcanza una cobertura de 0,957, próxima al 0,983 obtenido por la generación XML directa, y conserva una tasa de éxito semántico del 100 %. Los resultados de Opus 4.8 Medium se excluyen de esta tabla porque sus versiones cubren subconjuntos con distinta dificultad; compararlos introduciría un factor de confusión.

5.10 RQ6: Sensibilidad a la longitud de la descripción (planteamiento y trabajo futuro)

La sexta pregunta aborda el efecto de la longitud y del nivel de detalle de la descripción textual de entrada sobre la calidad del diagrama generado. A diferencia de las preguntas anteriores, aquí no se dispone de una matriz experimental controlada con versiones cortas y largas de un mismo proceso. Por ello, se plantea una discusión razonada y se reserva la evaluación empírica para el trabajo futuro (Capítulo 6).

La cuestión es relevante porque, en una aplicación real, los usuarios no describen los procesos con un estilo uniforme. Algunas entradas serán breves y ambiguas; otras incluirán excepciones, roles, datos, eventos y reglas de negocio con mucho detalle. El sistema debería ser capaz de aprovechar esa información adicional sin degradar su rendimiento por exceso de contexto.

Aunque no se comparan versiones cortas y largas de las descripciones, los datos del estudio ofrecen un indicio indirecto. El *system prompt* pasa de 11.186 *tokens* en v3.1 a 15.160 en v5 (Tabla 5.17). Al mismo tiempo, v5 mejora la cobertura semántica (0,957), pero reduce ligeramente la calidad estructural media frente a v3.1 (0,859 frente a 0,875). Ampliar el contexto de entrada, por tanto, no produce una mejora monotónica en todas las dimensiones.

Este efecto debe interpretarse teniendo en cuenta el tamaño absoluto de las entradas. Los modelos actuales ofrecen ventanas de contexto muy amplias, del orden de cientos de miles e incluso más de un millón de *tokens*. Un *system prompt* de unos 15.000 *tokens* ocupa, por tanto, una parte reducida del contexto disponible. En estas condiciones, la ligera disminución de la calidad estructural de v5 no parece deberse al agotamiento de la ventana. Resulta más plausible atribuirla al contenido de las instrucciones añadidas: un *prompt* con más reglas introduce también más restricciones que el modelo debe satisfacer al mismo tiempo. El coste principal del crecimiento del *prompt* es, así, económico, porque se procesan más *tokens* en cada solicitud y el *prompt caching* solo puede mitigarlo parcialmente.

La literatura reciente sobre el uso del contexto señala, no obstante, que en entradas muy largas la longitud efectivamente aprovechada suele ser inferior a la ventana nominal, y que la posición de la información influye en su utilización. Esta observación resulta más pertinente para descripciones extensas de procesos que para el *system prompt* empleado en este estudio. Como recomendación práctica, no como umbral derivado de la literatura, conviene que la descripción del proceso tenga una extensión moderada y una estructura clara. En una aplicación dirigida a usuarios no expertos, la interfaz debería guiar la introducción de roles, eventos, decisiones y excepciones, en lugar de depender únicamente de un bloque largo de texto libre. Determinar el nivel de detalle óptimo requiere el estudio controlado propuesto como trabajo futuro.

5.11 RQ7: Estabilidad entre ejecuciones

La séptima pregunta se centra en la estabilidad: qué ocurre cuando un mismo modelo ejecuta varias veces el mismo proceso con el mismo *prompt*. Para medirlo, se calcula la desviación típica media de *scored_mean* dentro de cada celda (*modelo*, *proceso*) a partir de las repeticiones R01, R02 y R03.

Modelo	Desviación típica intra-celda
GPT_5_5_THINKING	0,055
GEMINI_3_5_FLASH_THINKING	0,064
OPUS_4_7	0,077

Tabla 5.14: Estabilidad entre ejecuciones repetidas

Los tres modelos muestran una estabilidad razonable. GPT_5_5_THINKING obtiene la mejor, seguido de GEMINI_3_5_FLASH_THINKING y OPUS_4_7.

La estabilidad es un criterio relevante para una aplicación práctica. Un modelo con una media alta pero mucha variabilidad puede resultar menos predecible para el usuario final, sobre todo si en una ejecución genera un diagrama correcto y en otra omite una parte importante del proceso. Una desviación intra-celda menor indica, en cambio, que el sistema produce resultados más consistentes ante entradas equivalentes.

5.12 RQ8: Comparación zero-shot con generación BPMN-XML directa

Esta pregunta compara el DSL v3.1 con la generación directa de BPMN-XML para GPT-5.5. Ambos enfoques utilizan los mismos quince procesos y tres repeticiones, con 45 ejecuciones por representación. El XML devuelto por el modelo se conserva sin cambios en `output.bpmn`. No se aplican correcciones humanas, y cualquier *layout* automático se usa solo para renderizar, nunca para calcular M1–M10 ni la cobertura.

Enfoque	Éxito sem.	Calidad cond.	Calidad penal.	Cobertura
DSL v3.1	1,00	0,875	0,875	0,856
DSL v5	1,00	0,859	0,859	0,957
XML directo	0,87	0,927	0,804	0,983

Tabla 5.15: Robustez, calidad y cobertura de GPT-5.5 por representación

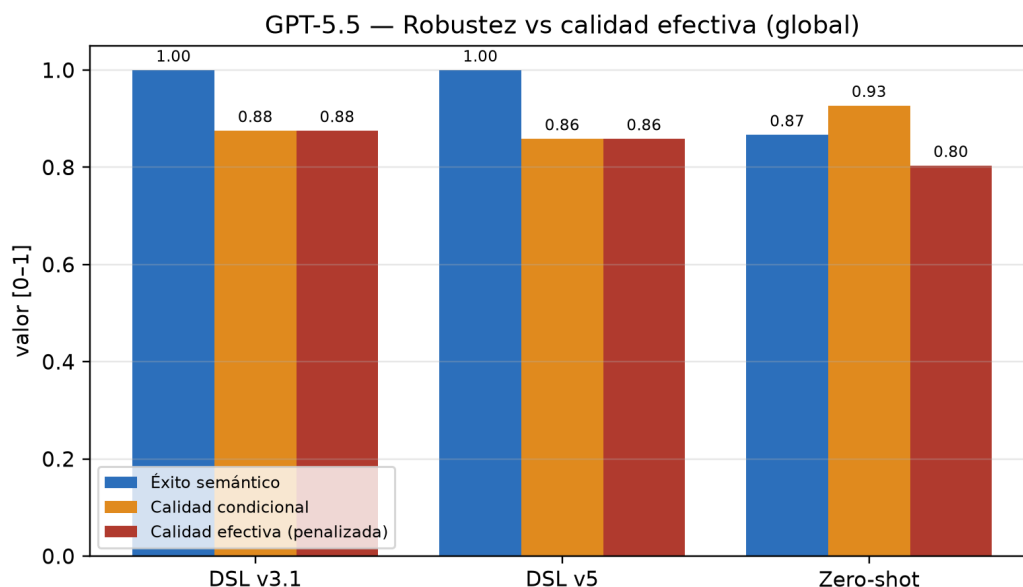


Figura 5.8: Robustez, calidad condicional y calidad efectiva en la comparación global

Si se consideran únicamente las salidas válidas, el XML directo alcanza una calidad media de 0,927 (Tabla 5.15 y Figura 5.8). Esa cifra excluye, sin embargo, el 13 % de ejecuciones que

no generan un artefacto semánticamente válido. Al asignar un cero a esos fallos, la calidad efectiva desciende a 0,804, por debajo del DSL v3.1 (0,875) y del DSL v5 (0,859). La aparente ventaja del XML directo en `scored_mean` condicional depende, por tanto, de seleccionar solo los casos que producen una salida válida.

Dificultad	DSL: éxito	DSL: calidad	XML: éxito	XML: cond. / penal.
Fácil	1,00	1,000	1,00	1,000 / 1,000
Media	1,00	0,851	0,85	0,925 / 0,788
Difícil	1,00	0,812	1,00	0,847 / 0,847
Estrés	1,00	0,846	0,33	0,813 / 0,271

Tabla 5.16: Comparación DSL v3.1 frente a XML directo por dificultad

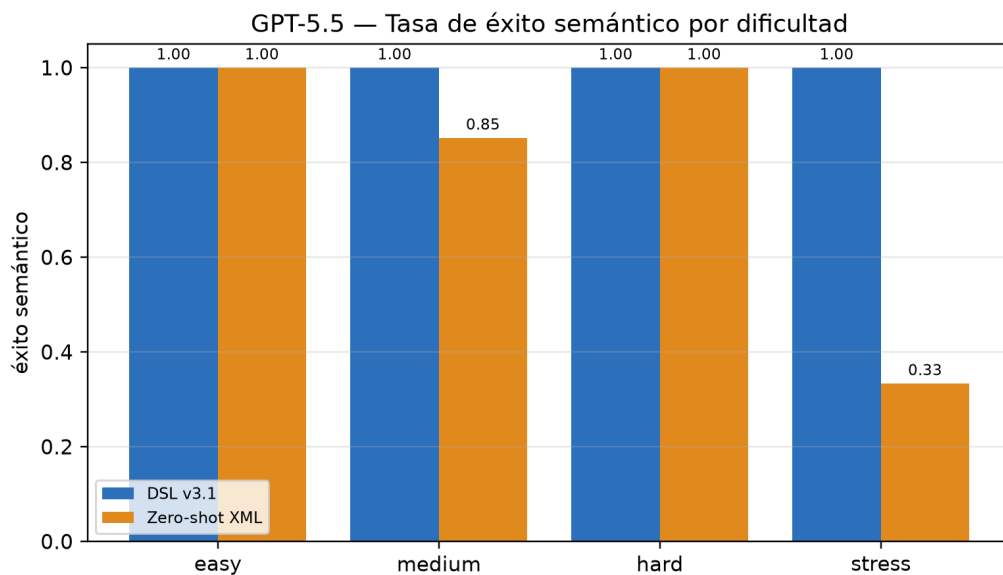


Figura 5.9: Tasa de éxito semántico del DSL y del XML directo por dificultad

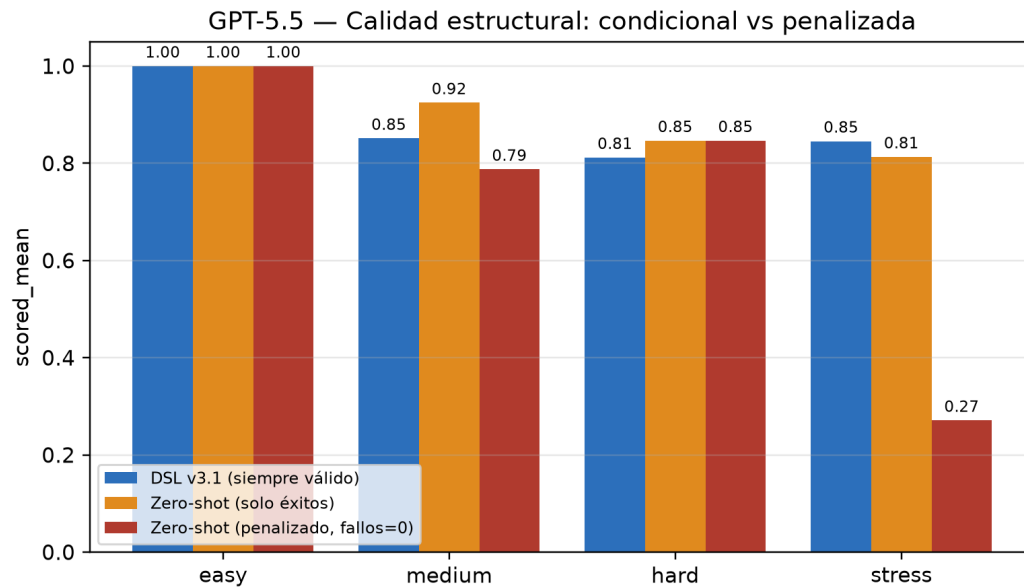


Figura 5.10: Calidad condicional y penalizada por dificultad

La estratificación por dificultad permite ver dónde aparece ese sesgo (Tabla 5.16 y Figuras 5.9 y 5.10). En los procesos de estrés, la calidad condicional de 0,813 del XML procede de una sola ejecución válida de las tres realizadas. Al incorporar los dos fallos, el valor baja a 0,271. El DSL mantiene un 100 % de éxito en todos los niveles y alcanza una calidad de 0,846 en estrés. Ambos enfoques son equivalentes en los casos fáciles y ofrecen resultados próximos en los difíciles, pero la robustez del XML cae con claridad en los procesos medios y, sobre todo, en los de estrés.

Enfoque	Tokens de entrada	Tokens de salida
DSL v3.1	11.186	197
DSL v4	13.519	252
DSL v5	15.160	263
XML directo	393	2.979

Tabla 5.17: Estimación textual de tokens por generación con GPT-5.5

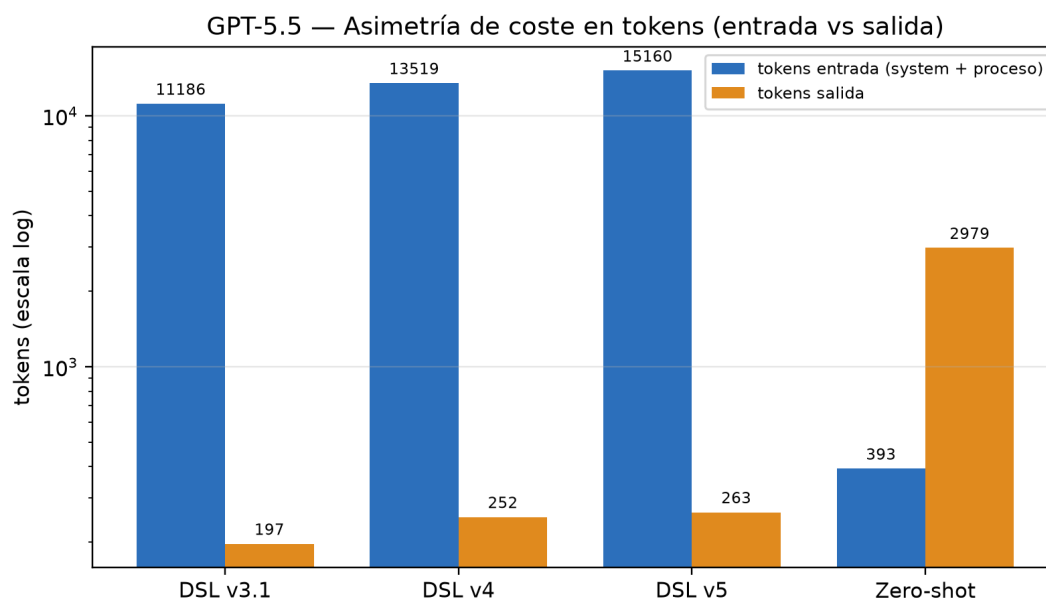


Figura 5.11: Asimetría entre tokens de entrada y salida según la representación

La distribución del coste textual es casi inversa en ambos enfoques (Tabla 5.17 y Figura 5.11). El DSL concentra los tokens en un *system prompt* extenso y genera una salida compacta, de unas 200–260 unidades. El XML directo parte de una entrada breve, pero produce alrededor de 3.000 tokens de salida. El *prompt* del DSL es fijo y podría almacenarse en caché mediante una API; la salida XML, en cambio, se genera y se factura de nuevo en cada ejecución. Estas cifras son estimaciones homogéneas obtenidas con `o200k_base`, no costes facturados, y excluyen el razonamiento oculto.

La comparación con generación mediante imagen directa y servidores MCP queda como trabajo futuro. Estas modalidades no generan necesariamente un BPMN-XML con el mismo grado de formalidad. Su evaluación deberá apoyarse en dimensiones comunes, como éxito, tiempo, tokens y cobertura de requisitos, sin aplicar métricas estructurales que no resulten pertinentes.

5.13 Validez externa: procesos reales en español

Además del corpus sintético, la evaluación incorpora procesos reales descritos en español. Esta extensión permite comprobar si el comportamiento observado en los procesos diseñados para el experimento se mantiene cuando la entrada se aproxima al uso previsto de la aplicación: descripciones naturales, terminología empresarial, posibles ambigüedades y estructuras propias de procesos reales.

Esta parte complementa el corpus sintético, pero no lo sustituye. El corpus sintético permite controlar la dificultad y las *features* esperadas; los procesos reales, en cambio, ayudan a examinar la representatividad y la utilidad práctica del enfoque.

Los criterios analizados comprueban si el modelo mantiene tasas aceptables de éxito semántico y de renderizado; si interpreta de forma razonable las lanes, los roles y los participantes; si traslada al diagrama las decisiones y excepciones descritas en lenguaje natural; si el resultado es comprensible para un usuario no técnico; y si aparecen problemas específicos del idioma, como etiquetas largas, acentos o nombres de roles en español.

Esta validación externa es importante porque la aplicación está pensada para personas que describen procesos en lenguaje natural, no para expertos que elaboran *prompts* optimizados. Si los resultados se mantienen en español y con procesos reales, la utilidad práctica del enfoque DSL queda mejor respaldada.

Debe entenderse, no obstante, como una prueba de plausibilidad externa, no como una evaluación estadística exhaustiva. El número de procesos reales es inferior al del corpus sintético y su objetivo principal es comprobar que las conclusiones no dependan únicamente de procesos diseñados de forma artificial.

5.14 Significancia estadística

La comparación de los tres modelos completos se realiza mediante una prueba de Kruskal–Wallis sobre `scored_mean`. Se utiliza esta prueba no paramétrica porque no exige asumir normalidad estricta en las distribuciones de las puntuaciones.

Prueba	Estadístico	p-valor	Interpretación
Kruskal–Wallis	H = 3,41	p = 0,181	No se detectan diferencias estadísticamente significativas entre modelos.

Tabla 5.18: Prueba global de significancia estadística

Las comparaciones par a par se realizan mediante Mann–Whitney U (Tabla 5.19):

Comparación	U	p-valor
OPUS_4_7 vs. GPT_5_5_THINKING	848	0,184
OPUS_4_7 vs. GEMINI_3_5_FLASH_THINKING	1049	0,771
GPT_5_5_THINKING vs. GEMINI_3_5_FLASH_THINKING	1236	0,072

Tabla 5.19: Comparaciones par a par mediante Mann–Whitney U

Aunque GPT_5_5_THINKING obtiene la media descriptiva más alta, la diferencia no alcanza significación estadística al nivel habitual de 0,05 (Tabla 5.18). En consecuencia, no puede afirmarse que exista un modelo claramente superior en calidad global. Los tres modelos completos son competitivos y ofrecen resultados medios cercanos.

El perfil de fallos aporta una lectura más útil que la media agregada. Algunos modelos tienden a omitir colaboraciones entre pools; otros presentan problemas de anclaje de fragmentos;

otros generan ramas que no alcanzan un final explícito. Para mejorar el sistema, conocer este patrón de errores resulta más informativo que una diferencia media sin significación estadística.

5.15 Amenazas a la validez

5.15.1 Validez interna

Una amenaza relevante es confundir los fallos del modelo con los del *renderer*. La separación entre `semantic_success` y `render_success` reduce este riesgo, porque distingue la capacidad del modelo para producir un DSL válido de la capacidad del sistema para materializarlo visualmente.

Otra amenaza procede de la saturación de algunas métricas estructurales en valores próximos a 1. M1, M2 y M3 son necesarias para comprobar la consistencia básica, pero discriminan poco entre modelos con buenos resultados. Por eso, las conclusiones consideran también M4, M8, la cobertura de *features* y los diagnósticos.

La comparación *zero-shot* incorpora dos amenazas adicionales. La primera es el sesgo de supervivencia: el `scored_mean` condicional excluye las salidas inválidas y puede favorecer al enfoque menos robusto. Para mitigarlo, se presentan conjuntamente la tasa de éxito, la calidad condicional y la calidad penalizada. La segunda amenaza afecta a la equivalencia instrumental. El DSL y el XML directo producen artefactos estructurados comparables, pero las futuras modalidades basadas en imagen o MCP podrían no alcanzar el mismo nivel de formalidad y requerir dimensiones comunes diferenciadas.

5.15.2 Validez de constructo

La evaluación no compara los diagramas con un modelo BPMN de referencia, lo que limita la posibilidad de afirmar equivalencia semántica completa. Aun así, el enfoque resulta adecuado para este trabajo: permite analizar numerosos experimentos de forma automática y reproducible, sin crear manualmente diagramas de referencia. También evita asumir que un único diagrama humano representa la única solución correcta, hipótesis discutible en BPMN cuando un mismo proceso admite varias representaciones válidas.

La cobertura de *features* esperadas actúa como una aproximación intermedia. No garantiza que el diagrama sea perfecto, pero permite detectar la omisión de elementos clave exigidos por el enunciado.

5.15.3 Validez externa

El corpus sintético permite controlar la dificultad, pero no representa toda la variedad de procesos reales. Por esa razón se incorpora una validación adicional con procesos reales en

español. En trabajos futuros será necesario ampliarla con más dominios, usuarios y estilos de descripción.

5.15.4 Validez estadística

El tamaño muestral de los modelos completos permite una comparación inicial, pero no garantiza potencia suficiente para detectar diferencias pequeñas. Los modelos exploratorios, además, cuentan con muestras reducidas y sus resultados no deben compararse directamente con los de los modelos completos.

El resultado más marcado del estudio requiere una precisión adicional. El nivel de dificultad de estrés está representado por un único proceso sintético, con tres repeticiones por modelo. La reducción acusada de calidad y éxito observada en ese nivel, tanto en RQ3 como en RQ8, debe leerse como un indicio del punto de ruptura del enfoque, no como una estimación generalizable. Para caracterizar con fiabilidad el comportamiento ante procesos de complejidad muy alta sería necesario ampliar el corpus con varios procesos de estrés.

5.16 Reproducibilidad

La evaluación se diseñó para que los resultados puedan regenerarse a partir de los artefactos del repositorio. La Tabla 5.20 enumera los elementos principales:

Artefacto	Función
<code>TFM-eval/results/experiments/results.csv</code>	Tabla principal con métricas por experimento.
<code>TFM-eval/results/experiments/results.json</code>	Resultados detallados, agregados y diagnósticos.
<code>TFM-eval/results/experiments/figures</code>	Figuras comparativas generadas por la evaluación. En esta memoria se han incorporado las figuras finales disponibles en <code>imagenes/</code> .
<code>docs/analisis-derivado/build_report.py</code>	Recalcula la calidad penalizada, la ablación por versión y las figuras comparativas.
<code>TFM-eval/src/bpmn_eval/usage.py</code>	Estima el consumo textual de tokens y documenta sus límites.
<code>docs/zero-shot-bpmn-xml-harness.md</code>	Especifica el protocolo reproducible para BPMN-XML directo.
<code>TFM-eval/notebooks/analysis.ipynb</code>	Cuaderno de análisis y visualización.
<code>TFM-eval/README.md</code>	Diseño metodológico y decisiones de la evaluación.

Tabla 5.20: Artefactos de reproducibilidad de la evaluación

Separar la generación de datos, el cálculo de métricas y la visualización facilita la auditoría del proceso. Si se incorporan nuevas ejecuciones, versiones de *prompt* o procesos reales, el *pipeline* puede regenerar las tablas y figuras sin modificar la lógica de evaluación.

5.17 Conclusiones de la evaluación

La evaluación permite extraer las siguientes conclusiones principales:

1. **El enfoque basado en DSL es robusto.** Los tres modelos completos alcanzan tasas elevadas de éxito semántico y de renderizado. En el conjunto final, los fallos de renderizado no tienen un peso relevante.
2. **Los modelos generan diagramas estructuralmente sólidos.** Las métricas de validez, conectividad y grado de los elementos presentan valores altos. Estos resultados indican que la arquitectura DSL + compilador determinista ayuda a mantener la corrección formal del BPMN generado.
3. **Las diferencias entre modelos existen, pero no son estadísticamente concluyentes en la media global.** GPT_5_5_THINKING alcanza la media descriptiva más alta y la menor variabilidad, aunque las pruebas estadísticas no permiten afirmar que sea significativamente superior a los demás modelos completos.
4. **La dificultad del proceso es el factor más determinante.** La calidad disminuye de forma clara desde los procesos fáciles hasta los de estrés. Los procesos que combinan colaboración, eventos, excepciones y múltiples requisitos simultáneos son los más difíciles para el sistema.
5. **La principal debilidad semántica es la colaboración entre participantes.** Las omisiones más habituales afectan a `message_flow`, `multiple_pools` y `send_receive`. Este resultado señala una línea concreta de mejora para futuras versiones del *prompt*, del DSL o del compilador.
6. **La versión del *prompt* modifica sustancialmente la cobertura.** v5 incrementa la cobertura hasta 0,957 en GPT-5.5 y 0,911 en Gemini 3.5. Esto indica que parte de las omisiones de colaboración registradas con v3.1 procedían de una limitación de las instrucciones.
7. **El XML directo combina alta calidad condicional con menor robustez.** Su media de 0,927 entre las ejecuciones con éxito se reduce a 0,804 al incorporar los fallos, frente al 0,875 del DSL v3.1. La diferencia se concentra en los procesos de estrés, donde el éxito del XML desciende al 33 %.
8. **La estabilidad entre ejecuciones es razonable.** Los modelos completos muestran una variabilidad moderada y GPT_5_5_THINKING es la opción más estable dentro del conjunto evaluado.
9. **La evaluación es extensible.** El *pipeline* admite nuevas versiones de *prompt*, variantes de descripción, modelos adicionales y procesos reales sin rediseñar la metodología.

En conjunto, los resultados respaldan la viabilidad del enfoque propuesto. La combinación de un LLM que genera una representación intermedia controlada y un compilador determinista que produce BPMN 2.0 permite obtener diagramas válidos, reproducibles y evaluables de

forma automática. La línea principal de trabajo futuro no pasa tanto por lograr la validez del XML BPMN, que el sistema ya resuelve de forma consistente, como por mejorar la fidelidad semántica en procesos colaborativos y de alta complejidad.

Conclusiones y Trabajo Futuro

Este capítulo recoge los principales hallazgos del trabajo y propone varias líneas de continuación. La lectura de los resultados se organiza alrededor de la hipótesis inicial: producir diagramas a través de una representación intermedia controlada y de un compilador determinista, en lugar de generar XML BPMN directamente, permite obtener modelos válidos, reproducibles y evaluables de forma automática.

6.1 Conclusiones

El trabajo ha incluido el diseño y la implementación de un sistema completo: un motor compartido, dos aplicaciones y un *pipeline* de evaluación, además de una evaluación empírica en condiciones controladas. A partir de los resultados obtenidos pueden formularse las siguientes conclusiones.

1. **El enfoque basado en DSL es robusto.** Los modelos evaluados alcanzan tasas altas de éxito semántico y de renderizado. En el conjunto homogéneo, ambas tasas coinciden; por tanto, los fallos restantes no se explican por el motor de maquetación, sino por aquellos casos en los que el modelo no produjo un DSL semánticamente válido. Separar la generación, asumida por el LLM, de la materialización gráfica, realizada por el compilador, reduce la fragilidad que aparece cuando se genera XML de forma directa.
2. **Los diagramas generados son estructuralmente sólidos.** Las métricas de validez XSD, conectividad y grado de los elementos se sitúan en valores próximos a la saturación. Cuando una salida supera el análisis semántico, el compilador tiende a producir diagramas correctamente conectados y consistentes desde el punto de vista formal. Las diferencias más relevantes aparecen en métricas más exigentes, como la completitud de las decisiones, las etiquetas y el emparejamiento de *gateways*.
3. **Las diferencias entre modelos no son concluyentes en la media global.** En la muestra analizada, GPT-5.5 Thinking presenta la media descriptiva más alta y la menor variabilidad. Sin embargo, ni la prueba de Kruskal–Wallis ($p = 0,181$) ni las comparaciones por pares permiten afirmar una superioridad estadísticamente significativa. Los tres modelos

con cobertura completa ofrecen resultados competitivos, de modo que la elección debería atender al perfil de errores relevante para cada proceso, no solo a la media agregada.

4. **La dificultad del proceso es el factor más determinante.** La calidad desciende al pasar de procesos fáciles y medios a procesos difíciles y de estrés. En estos últimos confluyen colaboración, eventos, excepciones y ramas simultáneas, lo que delimita el alcance práctico del enfoque actual. Como esta degradación no se debe a problemas de renderizado, cabe interpretarla como una consecuencia de la complejidad semántica.
5. **La principal debilidad es la colaboración entre participantes.** Las omisiones más frecuentes afectan a los flujos de mensaje, a la representación de múltiples *pools* y a las construcciones de envío y recepción. En algunos procesos multi-actor, la representación queda concentrada en un único participante y se diluye la diferencia entre el flujo de secuencia y la interacción mediante mensajes. Esta limitación marca el principal margen de mejora.
6. **El *prompt* determina una parte importante de la cobertura.** Con la versión v5, la cobertura media asciende a 0,957 en GPT-5.5 y a 0,911 en Gemini 3.5. Este resultado sugiere que varias de las omisiones de colaboración observadas con v3.1 procedían de las instrucciones proporcionadas, y que pueden reducirse sin modificar el compilador.
7. **La generación XML directa es menos robusta que la generación empleando DSL, pese a lo que su calidad condicional sugiere.** En GPT-5.5 alcanza un valor de 0,927 cuando solo se consideran las salidas válidas, pero baja a 0,804 al penalizar los fallos, frente al 0,875 obtenido por el DSL v3.1. En los procesos de estrés, la tasa de éxito de la generación directa desciende al 33 %, mientras que el DSL mantiene el 100 %.
8. **La estabilidad entre ejecuciones es razonable.** Las repeticiones realizadas con los mismos procesos y el mismo *prompt* muestran una variabilidad intra-celda moderada. Para una aplicación práctica, este comportamiento resulta favorable, porque facilita la obtención de resultados previsibles ante entradas equivalentes.
9. **El sistema es extensible y aplicable en un contexto real.** El *pipeline* permite incorporar nuevos modelos, versiones de *prompt*, descripciones y procesos sin rediseñar la metodología. Además, compañeros de SGS Productivity han probado la aplicación web en fase beta con procesos reales, lo que aporta plausibilidad externa y evidencia de utilidad más allá del corpus sintético.

La comparación principal se apoya en una matriz equilibrada de 135 experimentos, formada por quince procesos, tres repeticiones y tres modelos con 45 ejecuciones cada uno. El conjunto consolidado, por su parte, reúne 420 observaciones: 177 con DSL v3.1, 99 con v4, 99 con v5 y 45 con BPMN-XML directo. En cada pregunta se selecciona la configuración pertinente para evitar comparaciones de medias entre versiones, representaciones o niveles de cobertura distintos.

Vistos en conjunto, los resultados respaldan la viabilidad del enfoque. El modelo de lenguaje genera una representación intermedia controlada y, a partir de ella, un compilador determinista

produce BPMN 2.0 válido, reproducible y evaluable. Así, la dificultad principal deja de estar en la validez sintáctica o en el renderizado y se desplaza hacia la fidelidad semántica de los procesos colaborativos y de mayor complejidad.

La comparación presentada en la Sección 5.12 no permite concluir que una representación sea superior en todas las dimensiones analizadas. El XML directo logra una cobertura y una calidad más altas entre los casos válidos; el DSL, en cambio, alcanza una mayor tasa de éxito y una mejor calidad efectiva cuando se contabilizan los fallos. Por ello, la interpretación del resultado exige distinguir entre robustez, calidad condicional, calidad penalizada y cobertura.

6.2 Trabajo futuro

6.2.1 Mejora de la colaboración entre participantes

La mejora prioritaria consiste en reforzar la separación entre participantes y el uso de los flujos de mensaje. Para avanzar en esa dirección se plantean tres vías complementarias: refinar el *system prompt*, aprovechar el mapa explícito *pool-lane* de V5 y ampliar la validación semántica para que advierta cuándo un proceso multi-actor se ha concentrado en un único participante.

6.2.2 Refinamiento del *system prompt*

La matriz principal utiliza v3.1 como variable controlada, mientras que la ablación muestra una mejora clara de la cobertura con v5 respecto a v3.1 y v4. Como línea futura, será necesario identificar qué bloques de instrucciones explican esta mejora, comprobar si el efecto se mantiene en otros modelos y estudiar posibles costes secundarios, como una mayor longitud de entrada, diagramas de tamaño innecesario o nuevas normalizaciones.

6.2.3 Bucle cerrado de autorreparación

En la implementación actual, el LLM no recibe los diagnósticos generados por el *parser* o por el renderizador. Devolver estos errores al modelo, junto con una solicitud de corrección, permitiría recuperar parte de los casos que ahora fallan. Esta extensión podría integrar técnicas de *self-repair* y validación determinista, manteniendo un registro explícito de cada intento.

6.2.4 Ampliación de la comparación *zero-shot*

La comparación directa con BPMN-XML ya se ha realizado para GPT-5.5. Su ampliación debería incorporar otros modelos y modalidades basadas tanto en imagen como en servidores MCP. En estas últimas será preciso registrar la calidad visual, la cobertura semántica, el tiempo, los tokens, las iteraciones, los errores de herramienta y la riqueza de símbolos, diferenciando las métricas comunes de aquellas que requieren un artefacto BPMN-XML estructurado.

6.2.5 Evaluación semántica con referencia

La evaluación empleada no exige un diagrama de referencia. Esta decisión facilita la automatización y la reproducibilidad, aunque impide afirmar una equivalencia semántica completa. La anotación de un subconjunto por parte de expertos, complementada con una valoración de tipo *LLM-as-a-judge* o con un esquema basado en claridad, corrección y completitud, permitiría ampliar la dimensión cualitativa del análisis.

6.2.6 Verificación simbólica y orquestación

La incorporación de análisis de *soundness* mediante redes de Petri permitiría identificar bloqueos, ramas sin final y problemas de sincronización que las métricas actuales no cubren por completo. También podría estudiarse una arquitectura con agentes especializados y puertas de calidad orientada a planificar, generar, verificar y reparar procesos complejos.

6.2.7 Ampliación de la validez externa

La prueba beta debe extenderse a un mayor número de usuarios, dominios y estilos de descripción. En particular, conviene analizar el efecto de la longitud de la entrada y de su nivel de detalle. Los resultados de este análisis podrían transformarse en recomendaciones de interfaz que orienten la introducción de participantes, decisiones, eventos y excepciones.

6.2.8 Acceso mediante API e hiperparámetros controlados

La fase experimental se llevó a cabo mediante interfaces web públicas. Esta decisión favorece la validez ecológica, pero restringe el control sobre la *temperature*, el *top-p*, las semillas y las versiones exactas. Una réplica a través de API permitiría ampliar el tamaño de la muestra, fijar las configuraciones y analizar diferencias pequeñas con mayor potencia estadística.

6.2.9 Ampliación del conjunto de modelos

La ampliación más inmediata consiste en completar la matriz v3.1 de Opus 4.8 Medium, que actualmente cuenta con 24 de las 45 ejecuciones requeridas. Una vez cubiertos los quince procesos y las tres repeticiones, podrán recalcularse el ranking, los intervalos de confianza y las pruebas estadísticas a partir de cuatro modelos comparables.

En definitiva, el trabajo culmina en un sistema funcional, trazable y evaluable, acompañado de líneas de evolución bien delimitadas. El reto ya no consiste únicamente en producir BPMN válido, sino también en representar con fidelidad la colaboración y la complejidad. Con ese propósito, el *prompt*, el DSL, el compilador y el protocolo de evaluación pueden refinarse de manera independiente sobre una base común.

Organización del repositorio

Este capítulo no repite la arquitectura interna del motor, ya descrita en el Capítulo 4. Su objetivo es resumir cómo evolucionó la organización del proyecto y qué decisiones permitieron convertir el prototipo inicial en un sistema reproducible y utilizable.

Todo el código, los experimentos y la documentación están disponibles en el repositorio público que actúa como fuente de verdad del proyecto:

<https://github.com/ignagp34/BPMN-DSL-Monorepo> [31]. El proyecto se organiza como un único *monorepo*: todas las rutas citadas a lo largo de la memoria remiten a él.

7.1 Consolidación en un monorepo

El código se consolidó en un *monorepo* cuando la versión basada en el *system prompt* v4 alcanzó una estabilidad suficiente para utilizarse fuera del laboratorio. En ese momento, compañeros de SGS Productivity dedicados a la consultoría Lean y a la mejora de procesos empezaron a probar la aplicación con casos prácticos. Esa adopción temprana hizo necesario reunir en una única base versionada la aplicación destinada a los usuarios, el laboratorio experimental y el motor BPMN compartido. Así se evitó que la demostración y los experimentos evolucionaran desde implementaciones distintas.

7.2 Integración del paquete de evaluación

La segunda decisión organizativa relevante fue integrar `TFM-eval`, que inicialmente se había desarrollado como un repositorio independiente. Esta incorporación cerró el ciclo de reproducibilidad, ya que vincula en un mismo historial la versión del motor que genera los diagramas, los experimentos y el código encargado de calcular las métricas. De este modo, una modificación del compilador o del *layout* puede propagarse de manera controlada, permitir la regeneración de los resultados y conservar un registro de la configuración evaluada.

7.3 Documentación y automatización

Las carpetas dedicadas a la documentación y a la automatización completan esta estructura. `docs` recoge las decisiones técnicas y metodológicas, junto con los artefactos derivados del análisis, mientras que `scripts` reúne la orquestación de las tareas de renderizado y evaluación. En conjunto, el repositorio deja constancia de la evolución desde una prueba de concepto hasta una herramienta probada tanto en un contexto profesional como en un entorno experimental auditable.

Lista de acrónimos

API Application Programming Interface.

AST Abstract Syntax Tree (árbol de sintaxis abstracta).

BPM Business Process Management (gestión de procesos de negocio).

BPMN Business Process Model and Notation.

DAG Directed Acyclic Graph (grafo acíclico dirigido).

DSL Domain-Specific Language (lenguaje específico de dominio).

FSM Finite State Machine (máquina de estados finitos).

IA Inteligencia Artificial.

LLM Large Language Model (modelo de lenguaje de gran escala).

MAO Multi-Agent Orchestration (orquestración multiagente).

MCP Model Context Protocol.

NLP Natural Language Processing (procesamiento del lenguaje natural).

POWL Partially Ordered Workflow Language.

TFM Trabajo de Fin de Máster.

XML eXtensible Markup Language.

XSD XML Schema Definition (definición de esquema XML).

Bibliografía

- [1] H. Kourani, A. Berti, D. Schuster, and W. M. van der Aalst, “Promoai: Process modeling with generative ai,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 8708–8712.
- [2] K. Busch, A. Rochlitzer, D. Sola, and H. Leopold, “Just tell me: Prompt engineering in business process management,” *arXiv preprint arXiv:2304.07183*, 2023.
- [3] C. Lauer, P. Pfeiffer, A. Rombach, and N. Mehdiyev, “Assessing the business process modeling competences of large language models,” *arXiv preprint arXiv:2601.21787*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.21787>
- [4] A. Safan and J. Köpke, “BPMN-Chatbot++: LLM-based modeling of collaboration diagrams with data,” in *Business Process Management (BPM 2025)*. Springer, 2025.
- [5] N. Klievtsova, J.-V. Benzin, T. Kampik, J. Mangler, and S. Rinderle-Ma, “Process modeler vs. chatbot: Is generative ai taking over process modeling?” in *Business Process Management Workshops*. Springer, 2025, pp. 637–649.
- [6] L. F. Hörner, “Towards an llm-based conversational framework for business process modeling: Research approach and preliminary results,” in *Advanced Information Systems Engineering*. Springer, 2025, pp. 286–293.
- [7] L. Lin, Y. Jin, Y. Zhou, W. Chen, and C. Qian, “MAO: A framework for process model generation with multi-agent orchestration,” *arXiv preprint arXiv:2408.01916*, 2024.
- [8] P. Drakopoulos, P. Malousoudis, N. Nousias, G. Tsakalidis, and K. Vergidis, “Do LLMs speak BPMN? an evaluation of their process modeling capabilities based on quality measures,” *Computation*, 2026.
- [9] A. Ivanchikj, S. Serbout, and C. Pautasso, “Live process modeling with the BPMN Sketch Miner,” *Software and Systems Modeling*, vol. 21, no. 5, pp. 1877–1906, 2022.
- [10] C. Ziche and G. Apruzzese, “Llm4pm: A case study on using large language models for process modeling in enterprise organizations,” in *Lecture Notes in Business Information Processing 527 LNBIP*. Springer, 2024, pp. 472–483.

- [11] Metadev, “Metadev,” 2026, accedido el 19 de junio de 2026. [Online]. Available: <https://metadev.pro/>
- [12] M. Grohs, L. Abb, N. Elsayed, and J.-R. Rehse, “Large language models can accomplish business process management tasks,” in *Business Process Management Workshops*. Springer, 2024, pp. 453–465.
- [13] A. Rebmann, F. D. Schmidt, G. Glavaš, and H. Van Der Aa, “Evaluating the ability of llms to solve semantics-aware process mining tasks,” in *2024 6th International Conference on Process Mining (ICPM)*. IEEE, 2024, pp. 9–16.
- [14] T. Kampik, C. Warmuth, A. Rebmann, R. Agam, L. N. Egger, A. Gerber, J. Hoffart, J. Kolk, P. Herzig, G. Decker *et al.*, “Large process models: Business process management in the age of generative ai,” *arXiv preprint arXiv:2309.00900*, 2023.
- [15] S. Wenger, M. Spahic-Bogdanovic, and A. Martin, “Large language models for democratizing business process modeling: Bpmn model generation and style guide adherence,” in *Artificial Intelligence Research*. Springer, 2025, pp. 363–379.
- [16] J. T. Licardo, N. Tanković, and D. Etinger, “BPMN assistant: An LLM-based approach to business process modeling,” *Applied Sciences*, 2026.
- [17] N. Klievtsova, T. Kampik, J. Mangler, and S. Rinderle-Ma, “Conversationally actionable process model creation,” in *Cooperative Information Systems*. Springer, 2025, pp. 39–55.
- [18] F. Friedrich, J. Mendling, and F. Puhlmann, “Process model generation from natural language text,” in *Advanced Information Systems Engineering*. Springer, 2011, pp. 482–496.
- [19] P. Bellan, M. Dragoni, C. Ghidini, H. van der Aa, and S. P. Ponzetto, “Process extraction from text: Benchmarking the state of the art and paving the way for future challenges,” *arXiv preprint arXiv:2110.03754*, 2023.
- [20] J. T. Licardo, N. Tanković, and D. Etinger, “A method for extracting bpmn models from textual descriptions using natural language processing,” *Procedia Computer Science*, vol. 239, pp. 483–490, 2024.
- [21] Q. Nivon and G. Salaün, “Automated generation of bpmn processes from textual requirements,” in *Service-Oriented Computing*. Springer, 2025, pp. 185–201.
- [22] A. Berti, X. Wang, H. Kourani, and W. M. P. van der Aalst, “Specializing large language models for process modeling via reinforcement learning with verifiable and universal rewards,” *Process Science*, 2025.
- [23] L. F. Hörner, M. Möller, and M. Reichert, “Automatically generating BPMN 2.0 process models from natural language process descriptions: Challenges, framework, quality assessment,” *Business & Information Systems Engineering*, 2026.

- [24] P. K. Menaka Sekar, I. Matei, M. Zhenirovskyy, H. Y. Wong, S. Kohmura, S. Hotta, and A. Inomata, “Automated BPMN model generation from textual process descriptions: A multi-stage LLM-driven approach,” in *2026 IEEE International Systems Conference (Sys-Con)*. IEEE, 2026.
- [25] H. Kourani and S. J. van Zelst, “POWL: partially ordered workflow language,” in *Business Process Management: 21st International Conference (BPM 2023)*. Springer, 2023, pp. 92–108.
- [26] A. Brissard, F. Cuppens, and A. Zouaq, “What is the best process model representation? a comparative analysis for process modeling with large language models,” in *AI4BPM Workshop at BPM 2025*, 2025, preprint arXiv:2507.11356.
- [27] N. Ayoughi, D. Dewar, S. Nejati, and M. Sabetzadeh, “DSL or code? evaluating the quality of LLM-generated algebraic specifications: A case study in optimization at kinaxis,” in *International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP ’26)*, 2026.
- [28] H. Kourani, A. Berti, D. Schuster, and W. M. van der Aalst, “Process modeling with large language models,” in *Enterprise, Business-Process and Information Systems Modeling*. Springer, 2024, pp. 229–244.
- [29] J. Neuberger, L. Ackermann, H. van der Aa, and S. Jablonski, “A universal prompting strategy for extracting process model information from natural language text using large language models,” in *Conceptual Modeling*. Springer, 2024, pp. 38–55.
- [30] Emergent Mind, “MOSAIC: Multi-agent orchestration,” 2026, updated 13 January 2026. [Online]. Available: <https://www.emergentmind.com/topics/multi-agent-orchestration-mosaic>
- [31] I. González Peris, “BPMN-DSL-Monorepo: motor, laboratorio experimental y pipeline de evaluación para la generación de BPMN 2.0 mediante un DSL intermedio,” <https://github.com/ignagp34/BPMN-DSL-Monorepo>, 2026, repositorio de software del proyecto (fuente de verdad). Última consulta: junio de 2026.
- [32] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [33] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.
- [34] W. R. Shadish, T. D. Cook, and D. T. Campbell, *Experimental and Quasi-Experimental Designs for Generalized Causal Inference*. Houghton Mifflin, 2002.
- [35] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, “Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting,” in *International Conference on Learning Representations (ICLR)*, 2024.

- [36] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [37] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing incrementally for constrained auto-regressive decoding from language models,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [38] J. Cardoso, “Control-flow complexity measurement of processes and weyuker’s properties,” in *Proceedings of World Academy of Science, Engineering and Technology*, vol. 8, 2005, pp. 213–218.

Apéndices

Material adicional y artefactos de reproducibilidad BPMN

A.1 Especificación del DSL y *system prompt*

La gramática completa del DSL textual se conserva en el siguiente fichero:

`apps/tfm-lab/prompts/system/bpmn_sketch_miner_system_prompt_v3_1.md`

Este documento cumple una doble función: especifica el lenguaje y proporciona el condicionamiento *few-shot* del modelo. En los experimentos correspondientes a la matriz principal, la versión v3.1 se utiliza como variable controlada.

El artefacto consta de 1201 líneas y presenta la siguiente huella SHA-256 (calculada sobre el fichero tal como se conserva en el repositorio):

5A574F36BE54EBD8B6E22F65D747974265616B15A547B893170365995CCC247F

Sus bloques principales documentan:

- el formato de salida y las reglas fundamentales del DSL;
- tareas, roles, *pools*, *lanes*, compuertas, eventos y elementos de datos;
- paralelismo, decisiones y modelado mediante trazas;
- fragmentos y restricciones de anclaje, incluidos los antipatronos AP-6 y AP-7;
- ejemplos completos *few-shot*; y
- una lista final de comprobación antes de devolver la respuesta.

Las referencias incluidas en la metodología a las secciones internas del *prompt*, los antipatronos y sus ejemplos remiten a este artefacto electrónico versionado. Aunque las versiones posteriores se encuentran en `apps/tfm-lab/prompts/system`, no forman parte de los resultados obtenidos con la matriz v3.1.

A.2 Resultados completos y trazabilidad experimental

Los resultados consolidados se proporcionan en `TFM-eval/results/experiments/results.csv` y `TFM-eval/results/experiments/results.json`. El primer fichero facilita el análisis tabular, mientras que el segundo mantiene la estructura completa de las métricas y los diagnósticos. Además, en la carpeta experimental correspondiente a cada ejecución se conservan la entrada, la salida normalizada, el diagrama, los metadatos y el resultado de la validación.

Conjunto	N	Uso en la memoria
Matriz principal equilibrada	135	Tres modelos, quince procesos y tres repeticiones; sustenta la comparación estadística principal.
Resultados exploratorios	33	Modelos o configuraciones con cobertura incompleta; se conservan para trazabilidad, pero no se mezclan con el ranking.
Conjunto consolidado	168	Filas evaluadas y reproducibles de <code>results.csv</code> , todas asociadas a SYSV31.
Inventario bruto del repositorio	203	Carpetas generadas durante el desarrollo, incluidas pruebas, versiones de <i>prompt</i> y ejecuciones pendientes de consolidación.

Tabla A.1: Relación entre los conjuntos experimentales utilizados.

Las tablas completas de cada experimento pueden regenerarse mediante `scripts/run-evaluation.ps1`. Para fijar una ejecución deben registrarse la versión del motor, la fecha, el modelo, el proceso, la repetición y la versión del *system prompt*. Entre las métricas principales se incluyen `semantic_success`, `render_success`, `scored_mean`, M1–M5, M8 y la cobertura de las características esperadas.

A.3 Ejemplo de extremo a extremo

Como ejemplo de aplicación, se incluye el proceso de una planta de tratamiento de residuos. El artefacto editable correspondiente se adjunta como material complementario de esta memoria, mientras que la Figura A.1 muestra su representación exportada. La extensión `.bpm` es la adecuada, ya que corresponde a un paquete de proyecto del modelador que contiene el diagrama editable y no a un fichero BPMN-XML plano con extensión `.bpmn`.

Este caso permite examinar la parte final de la cadena de extremo a extremo: la descripción del proceso, la representación intermedia, el modelo BPMN editable y la visualización final. Su tamaño también permite observar la dificultad que supone mantener participantes, *lanes*, eventos, decisiones y múltiples ramificaciones dentro de un único diagrama. Por ese tamaño,

para visualizar mejor el diagrama se recomienda abrir las versiones renderizadas a tamaño completo (SVG y PNG) que acompañan a cada ejecución en el repositorio del proyecto, en [apps/tfm-lab/prompts/experiments](https://github.com/tfm-lab/prompts/experiments) [31].

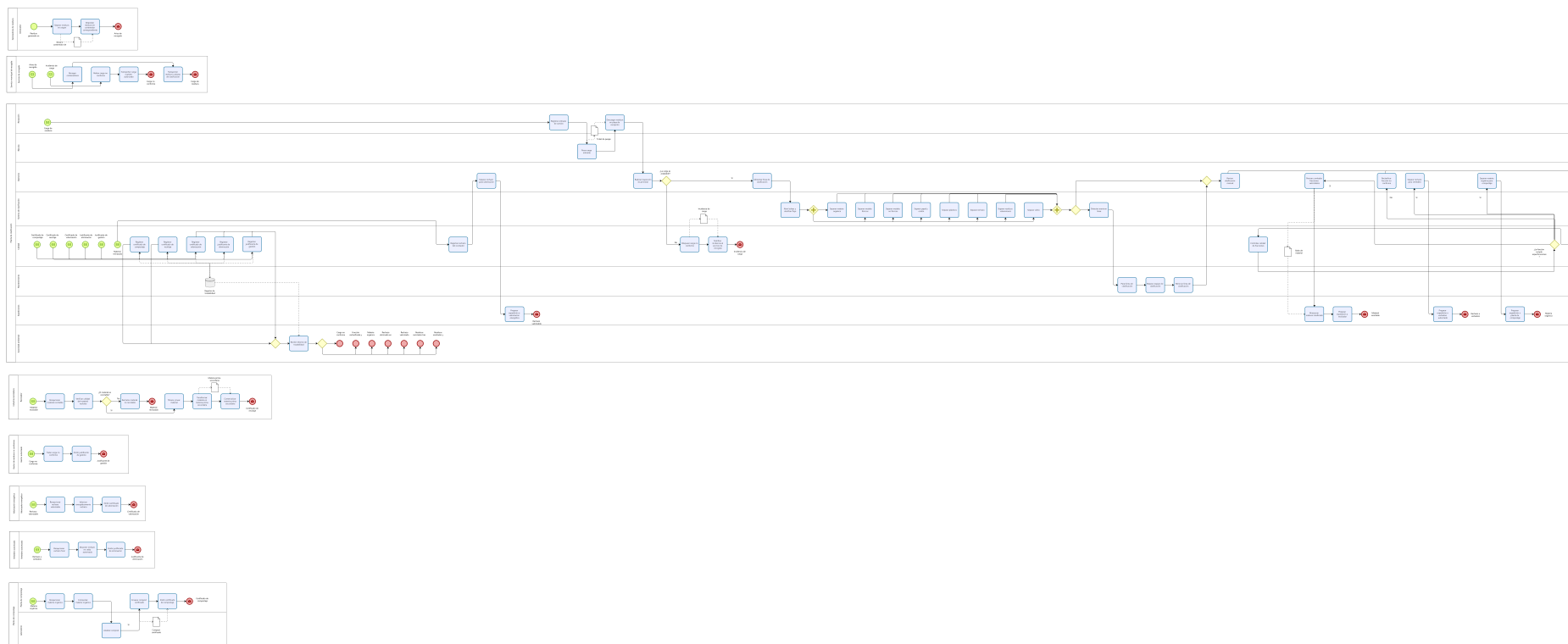


Figura A.1: Modelo BPMN de la planta de tratamiento de residuos.

A.4 Galería de diagramas *canon*

Los diagramas *canon* se utilizan como casos de regresión de extremo a extremo. Sus fuentes están disponibles en [packages/bpmn-core/tests/fixtures](#), y cualquier modificación del motor debe preservar las propiedades visuales y estructurales que estos casos representan.

Caso	Proceso	Aspectos comprobados
<i>canon-1</i>	Preparación de hamburguesa	Roles, estaciones paralelas, temporizador y eventos explícitos de inicio y fin.
<i>canon-2</i>	Vuelo entrante	Participantes, eventos y continuidad de flujos entre actividades relacionadas.
<i>canon-3</i>	Cinta transportadora	Enrutado ortogonal y separación entre ramas y elementos del proceso.
<i>canon-4</i>	Compostaje	Decisiones, uniones y legibilidad en procesos con varias alternativas.
<i>canon-5</i>	Boda	<i>Lanes</i> , colaboración, ramas y colocación de uniones en un proceso de mayor tamaño.

Tabla A.2: Diagramas canónicos utilizados como pruebas de regresión.

En la versión electrónica del proyecto, cada entrada DSL se conserva junto con el render generado. Esta correspondencia permite comparar la fuente textual con el resultado gráfico sin tener que interpretar manualmente el historial de cambios.

A.5 Catálogo de códigos de diagnóstico

Los diagnósticos permiten determinar si un fallo procede del análisis del DSL, del modelo BPMN o del *layout*. La Tabla A.3 presenta los errores de entrada, y la Tabla A.4 resume los hallazgos obtenidos en las fases posteriores.

Código	Interpretación
LEX-1, PARSE-1	El texto no puede tokenizarse o agruparse conforme a la gramática.
AP-6, AP-7	Violación de las reglas de construcción o de frontera de fragmentos.
ANCHOR-MISSING	No se encuentra el ancla necesaria para continuar una traza.
INLINE-COMMENT	Se detecta un comentario en una posición no admitida.
FRAGMENT-OPEN	Un fragmento permanece abierto al finalizar el análisis.
FRAGMENT-DUP	Se declara más de una vez el mismo fragmento o ancla.
DANGLING-QUESTION	Una decisión no dispone de ramas válidas asociadas.

Tabla A.3: Diagnósticos del analizador léxico y sintáctico.

Código o familia	Origen	Interpretación
IMPLICIT_START_EVENT, IMPLICIT_END_EVENT, IMPLICIT_BOUNDARY_END_EVENT	Modelo	Elementos sintetizados por el motor para completar una traza.
MISSING_EXPLICIT_START_EVENT, MISSING_EXPLICIT_END_EVENT	Modelo	El DSL no declara explícitamente el inicio o el final.
SEQUENCE_FLOW_INVALID_REFERENCE	Modelo	Un flujo de secuencia apunta a un elemento inexistente.
MESSAGE_FLOW_INVALID_REFERENCE, MESSAGE_FLOW_INVALID_ENDPOINT	Modelo	Referencia o extremo inválido en un flujo de mensaje.
MESSAGE_FLOW_WITHIN_POOL	Modelo	Se utiliza un flujo de mensaje dentro del mismo participante.
PPOOL_BOUNDARY_CONTROL_FLOW	Modelo	Un flujo de control atraviesa indebidamente el límite de un participante.
PPOOL_MAP_DUPLICATE_LANE, PPOOL_MAP_LANE_UNASSIGNED	Modelo	Una <i>lane</i> está duplicada o no se ha asignado en el mapa explícito.
PPOOL_MAP_NAME_COLLISION, PPOOL_MAP_UNKNOWN_LANE	Modelo	Colisión de nombres o referencia a una <i>lane</i> desconocida.
LAYOUT_ARTIFACT_TOO_FAR	<i>Layout</i>	Un artefacto queda excesivamente alejado de su elemento asociado.
LAYOUT_ELEMENT_OVERLAP	<i>Layout</i>	Dos elementos gráficos se solapan.
LAYOUT_UNREADABLE_LABEL	<i>Layout</i>	Una etiqueta no dispone de una colocación legible.
LAYOUT_XML_PARSE_ERROR	<i>Layout</i>	No se puede analizar el XML durante la fase gráfica.

Tabla A.4: Diagnósticos del modelo y del subsistema de *layout*.

A.6 Evolución del *system prompt*

Versión	Uso	Cambios principales
v3.1	Matriz experimental principal	Gramática, antipatrones, ejemplos <i>few-shot</i> y lista de comprobación.
v4	Ablación posterior	Consejos adicionales de legibilidad y mejoras en la generación de eventos de fin.
v4.1	Refinamiento de v4	Ajustes derivados de las pruebas sobre los casos canónicos.
v5	Desarrollo posterior	Mapeo explícito <i>pool-lane</i> mediante el bloque <code>== pools ==</code> .

Tabla A.5: Evolución funcional del *system prompt*.

Los artefactos almacenados en [apps/tfm-lab/prompts/system](#) presentan los siguientes tamaños y huellas SHA-256 (calculadas sobre los ficheros tal como se conservan en el repositorio):

v3.1 1201 líneas: 5A574F36BE54EBD8B6E22F65D747974265616B15A547B893170365995CCC247F

v4 1347 líneas: FF470D282CD40DCBC31E9B313388B2438B3823C19C7526362631ADA4CDD40E50

v5 1408 líneas: 70FCF2DB9276A3E8B59E2125779D6767DCD25C0177F5F4F74FA7B62E9A82BC16

La matriz consolidada actual emplea únicamente v3.1. Las versiones posteriores permiten documentar la evolución del sistema, aunque no sirven de base para las conclusiones estadísticas principales.

A.7 Conjunto de procesos sintéticos

El corpus experimental está formado por quince procesos, identificados como SYN001–SYN015. La especificación de cada uno se conserva en [apps/tfm-lab/prompts/processes](#) e indica tanto su nivel de dificultad como las características BPMN esperadas.

Grupo	Dificultad	Características predominantes
SYN001–SYN003	Fácil	Procesos lineales, eventos básicos y ramificaciones sencillas.
SYN004–SYN012	Media	Decisiones, paralelismo, colaboración, <i>lanes</i> y elementos de datos.
SYN013–SYN014	Difícil	Eventos avanzados, excepciones, cancelaciones y colaboración compleja.
SYN015	Estrés	Combinación simultánea de múltiples <i>lanes</i> , compuertas, flujos de mensaje, datos y excepciones.

Tabla A.6: Organización del corpus sintético por dificultad.